IMPERIAL

Imperial College London
Department of Mathematics

Dynamic hierarchical stochastic block models

Jakob Creutzig

CID: 02543387

Supervised by Francesco Sanna Passino

August 28, 2025

Submitted in partial fulfilment of the requirements for the
MSc in Machine Learning and Data Science at Imperial College London

The work contained in this thesis is my own work unless otherwise stated.

Signed: Jakob Creutzig Date: August 28, 2025

Acknowledgements

I am grateful to my family who had my back as always, to my supervisor for several beneficial interactions, and to Norges Bank for supporting my education.

Abstract

We extend a known model for hierarchical SBM random graphs to dynamic hierarchical networks. Our new model utilizes the hierarchical setup to split static and dynamic community allocation aspects, also extending known dynamic stochastic block models. Further we establish an estimation algorithm for our model utilizing the recently introduced technique of unfolded adjacency embedding. A novel component for temporal stability scoring allows to efficiently separate temporally stable super-communities from dynamically evolving sub-communities. We test this estimation method on simulated data and find our new algorithm to perform well over a large range of parameters. Further we apply it to bike sharing data from the Transport for London cycling infrastructure dataset, where our method is capable to infer from connectivity patterns a community allocation consistent with geospatial features of the network.

1 Introduction

Dynamic networks are networks which randomly change as time passes. Examples of such networks abound, from brain cell interactions through time over bitcoin transactions and communications in computer networks, to bike rental activity and social media interactions. Dynamic networks provide both a possibility to use a richer data setup for better estimation of underlying structures than classical random graphs, and to enable predictions for temporal changes of said underlying structures.

Statistical models for such networks add a temporal aspect to the modeling and study of random graphs and can be interpreted as generalizations of random graph models. A popular model for random graphs are latent embedding models introduced in Hoff et al. (2002) where nodes are represented by vectors $\mathbf{x}_i \in \mathbb{R}^d$ and the probability of two nodes to form a connection is determined by some kernel function κ via

$$\mathbb{P}\left(\text{"}i \text{ and } j \text{ connected"}\right) = \kappa(\mathbf{x}_i, \mathbf{x}_j) \ .$$

The generalization of these models to the dynamic case can be pursued by making either the proximity measure κ dynamic, the embedding $\mathbf{x}_i$, or both. A recent achievement (Jones and Rubin-Delanchy, 2020) was the introduction of multilayer random dot graph models. These models generalize several established latent space models to dynamic networks and provide a novel algorithm for joint estimation of latent embeddings as time-dependent paths in the latent space, the unfolded spectral adjacency embedding.

One particular subclass of models for random graphs, the stochastic block models, group nodes into communities and specify connection probabilities between nodes based on co-membership in said communities. A generalization of SBMs are hierarchical models which assume the existence of larger super-communities, which are split into smaller sub-communities (e.g. interest communities in social media on football, tennis and chess could all belong to a sports interest super community), and that connection probabilities depend on the tiered co-membership within super-community and/or sub-community. The paper Lyzinski et al. (2016) models a hierarchical stochastic block model for a random graph by assigning a node first randomly to a super-community and thereafter randomly picking a (sub-)community vector within that super-community. A two-step estimation procedure for detecting hierarchical communities in random graphs was also established by the authors, which also classifies super-communities according to motif.

The above approaches can model and estimate either non-hierarchical stochastic block

models in a dynamic network, or hierarchical models in a random graph. However, hierarchical structures can occur also in settings where a strong temporal element is present, a setup which is covered by neither of the two established approaches. As an example, one important motivation cited in Lyzinski et al. (2016) is the cortical column conjecture which suggests a hierarchical organization of neurons. Related research on neural activity on visual stimuli also suggests the existence of functional hierarchies besides anatomical ones, with communication clusters spanning multiple brain regions (Jia et al., 2022; Siegle et al., 2021). However, many neural activities exhibit significant non-stationarity (e.g. reflecting learning) and dynamic networks have been proposed as a natural model (Humphries, 2017). This suggests that dynamic hierarchical models which allow for at least partial dynamics in community memberships over time, could be beneficial for the understanding of structured neuron activity over time. Another example where a hierarchical model would naturally show dynamic behaviour would be a university computer network separated into super-communities according to e.g. organizational units, and thereafter internally into communication clusters within each super-community.

We establish a new model for *dynamic hierarchical stochastic block models*, which assumes a static assignment of nodes to a super-community, and allows nodes to change membership in one of several sub-communities over time. We also present a new algorithm for estimation of super- and (dynamic) sub-community membership in such models. Our algorithm follows the spirit of the approach in Lyzinski et al. (2016) for hierarchical community detection, but utilizes key methods from Jones and Rubin-Delanchy (2020) for generalizing them to the dynamic case. It also introduces a novel component of temporal stabilization which significantly boosts the classification performance in empirical tests.

The remainder of the report is organized as follows. In the Methods section, we introduce random graph models and adjacency spectral embedding, and further present known models for dynamic networks, especially the multilayer random dot product model. In the Model section, we present our new dynamic hierarchical degree corrected stochastic block model and then establish an estimation algorithm for super- and sub-community membership. In the Results section, we demonstrate and discuss the performance of our algorithm on simulated data over a range of parameters and against a naive variant of the algorithm, and finally present an application to the bike rental data from the Transport for London cycling infrastructure dataset. A discussion of our findings concludes the report.

1.1 Notation

We set $\mathbb{N} = \{1, 2, \ldots\}$, and for $n \in \mathbb{N}$ we let $[n] := \{1, \ldots, n\}$. We denote the real numbers by $\mathbb{R}$ and the standard Euclidean vector spaces $\mathbb{R}^d$. The indicator function $x \mapsto \mathbf{1}[x \in E]$ is one on the set/event E, and 0 otherwise. With slight abuse of notion we also use this for logical checks, e.g. $\mathbf{1}[x > 0]$.

We use upper and lower case normal letters for real numbers (e.g. $a \in \mathbb{R}$, $T \in \mathbb{N}$), bold lower case letters for vectors[1] (e.g. $\mathbf{c} \in \mathbb{R}^d$), and upper case bold letters for matrices (e.g. $\mathbf{A} \in \mathbb{R}^{n \times m}$). For a matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$, we write its transpose as $\mathbf{A}^\top \in \mathbb{R}^{m \times n}$. The entries of a matrix are denoted by corresponding lower case letter with sub-indexing (e.g. $\mathbf{A}$ as above has entries $a_{i,j}$ for $i \in [n], j \in [m]$). For a diagonal matrix $\mathbf{\Sigma}$ with non-negative entries, $\mathbf{\Sigma}^{1/2}$ denotes the diagonal matrix with the diagonal being the element-wise square root of $\mathbf{\Sigma}$.

In this work, we will need to consider series of matrices and vectors which will be indexed both by time $t \in [T]$ and by some category $k \in [K]$. In order to avoid overly bulky indexation, we use the convention that $\mathbf{A}^t$ indicates a time series[2], while sub-indices of the form $\mathbf{A}_{(k)}$ indicates split per category. A simple sub-index like $\mathbf{x}_i$ is a per-node indexation (we will clarify the meaning of each indexation in the text).

We often want to 'unfold' vectors or matrices into a single matrix. Say we have a time series of matrices $\mathbf{X}^1, \ldots \mathbf{X}^T$ of common dimensions $n \times d$. We then set

$$\mathbf{X} = \left[\mathbf{X}^1 | \cdots | \mathbf{X}^T\right]$$

as the matrix of dimension $n \times Td$ with entries $x_{i,(t-1)d+j} := x^t_{i,j}$ for $i \in [n], j \in [d], t \in [T]$. We also use this notation to split an $n \times dT$ matrix into T matrices of dimension $n \times d$ in the opposite way. For a given matrix $\mathbf{X}$, we allow for a slight abuse of notion, and consider $\mathbf{x}_i$ (being column vectors) to be its transposed row vectors, such that

$$\mathbf{X} = \left[\mathbf{x}_1 | \cdots | \mathbf{x}_n\right]^\top .$$

For a series of vectors say $\mathbf{x}_1, \ldots \mathbf{x}_n \in \mathbb{R}^d$, we denote their Gram matrix as

$$\mathrm{Gram}\left(\mathbf{x}_1, \ldots \mathbf{x}_n\right) := \left(\mathbf{x}_i^\top \mathbf{x}_j\right)_{i,j \in [n]} \in \mathbb{R}^{n \times n} .$$

[1] Vectors are always interpreted as column vectors.

[2] Since the only power of matrices we use is the square root, this notation should not lead to confusion with the usual power of a matrix.

With $\mathbf{X} := [\mathbf{x}_1|\cdots|\mathbf{x}_n]^\top \in \mathbb{R}^{n\times d}$, this can also be written as

$$\operatorname{Gram}(\mathbf{x}_1,\ldots,\mathbf{x}_n) = \mathbf{X}\mathbf{X}^\top .$$

We use script font for sets, i.e. $\mathcal{X} \subseteq \mathbb{R}^d$, and fraktura font like $\mathfrak{F}$ for probability measures. For a set $\mathcal{X} \subseteq \mathbb{R}^d$ and some element $\mathbf{c} \in \mathcal{X}$, the measure $\delta_{\mathbf{c}}$ denotes the atomic Dirac measure concentrated in $\mathbf{c}$.

On some occasions, we will need to consider the behaviour of algorithms on graphs as the number of nodes tends to ∞. We will use the notation $\mathbf{X}^{\{n\}}$, $\hat{\tau}^{\{n\}}$ etc to indicate sequences of objects related to a sequence of graphs with growing node count n, and will also clearly indicate in the text where we apply this convention.

2 Methods

In this section we review known results from the literature. We will first introduce and briefly discuss random graph models and their estimators, with an emphasis on community estimation in block models. Then, we will discuss models for dynamic networks, essentially the multi layer random dot product model, and the unfolded spectral adjacency embedding as a core algorithm for dynamic RDPGs and in particular dynamic SBMs.

2.1 Random graph models

A graph is a tuple $G = (V, E)$ of a nonempty set V and a subset $E \subseteq V \times V$ of tuples (v, w) (indicating that v connects to w), representing a form of relationship. We assume that $V = [n]$ and encode E in the binary *adjacency matrix* $\mathbf{A} = (a_{i,j})_{i,j\in[n]}$ where

$$a_{i,j} = 1 \qquad \text{iff} \qquad (i,j) \in E .$$

We require that $a_{i,i} = 0$ for all i (no self loops) and $\mathbf{A} = \mathbf{A}^\top$ (undirected graph). For $W \subseteq V$, $\operatorname{Sub}(G, W) := (W, E \cap (W \times W))$ is the subgraph with nodes W.

A *random graph* is given by a fixed node set $V = [n]$ and a randomly generated adjacency matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$. We will focus on *edge-independent* random graphs: A *graph probability*

matrix is a symmetric matrix $\mathbf{P} = (p_{i,j})_{i,j\in[n]} \in [0,1]^{n\times n}$. Given such $\mathbf{P}$, we can generate a random variable $\mathbf{A} = (a_{i,j})_{i,j\in[n]}$ by generating

$$a_{i,j} \sim \text{Bernoulli}(p_{i,j}) \text{ independently}, \quad 1 \le i < j \le n,$$

and setting $a_{i,j} := a_{j,i}$ for $j > i$, and $a_{i,i} := 0$. We write in short $\mathbf{A} \sim \mathbf{P}$ and say $\mathbf{P}$ generates $\mathbf{A}$. If $\mathbf{P}$ itself is generated randomly, then the above random generation model is understood conditional on $\mathbf{P}$. In the following, we will differentiate models based on the manner $\mathbf{P}$ itself is generated.

A popular modern approach is to assume that one can find an embedding of the vertices $1,\dots,n$ into a suitable d dimensional set $\mathcal{X} \subseteq \mathbb{R}^d$ and a symmetric kernel function $\kappa : \mathcal{X}\times\mathcal{X} \to [0,1]$. If for vertices i,j we denote their embedding with $\mathbf{x}_i, \mathbf{x}_j$, then

$$p_{i,j} := \kappa(\mathbf{x}_i, \mathbf{x}_j)$$

defines a graph probability matrix $\mathbf{P} = (p_{i,j})_{i,j\in[n]}$, and $\mathbf{A} \sim \mathbf{P}$ is said to be generated by the latent embedding $\mathbf{X} = [\mathbf{x}_1|\cdots|x_n]$.

Usually one assumes that $\mathbf{x}_i$ are generated as i.i.d. realizations $\mathbf{x}_i \sim \mathfrak{F}$ of some suitable[3] measure $\mathfrak{F}$. This is known as the *latent embedding model* (introduced in Hoff et al. (2002)), and is specified by $\mathfrak{F}$ and κ.

In (Nickel, 2008; Young and Scheinerman, 2007), the *random dot product graph (RDPG)* was introduced, which is a latent embedding model with simply

$$p_{i,j} = \kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j, \qquad \text{i.e.} \qquad \mathbf{P} = \text{Gram}\,(\mathbf{x}_1 \dots, \mathbf{x}_n) = \mathbf{X}\mathbf{X}^\top. \tag{1}$$

In a small abuse of notion, we write $\mathbf{A} \sim \text{RDPG}\,(\mathfrak{F})$ to indicate that $\mathbf{A} \sim \mathbf{P}$ where $\mathbf{P}$ is generated by (1) from $\mathbf{x}_i \sim \mathfrak{F}$ i.i.d. .

Any RDPG model leads to a positive semi-definite probability matrix $\mathbf{P}$. On the other hand, each positive semi-definite probability matrix $\mathbf{P}$ can be factorized as described in (1) with $d = \text{rank}(\mathbf{P})$, using e.g. the Cholesky decomposition $\mathbf{P} = \mathbf{\Lambda}\mathbf{\Lambda}^T$.

An important property of RDPG models is *orthogonal invariance*, i.e. for two measures $\mathfrak{F}$ and $\mathfrak{F}'$ such that there is an orthogonal matrix $\mathbf{O} \in \mathbb{R}^{d\times d}$ with $\mathfrak{F}' = \mathfrak{F} \circ \mathbf{O}^{-1}$, then two $n\times n$ random adjacency matrices $\mathbf{A} \sim \text{RDPG}\,(\mathfrak{F})$ and $\mathbf{A}' \sim \text{RDPG}\,(\mathfrak{F}')$ will have identical distribution. One consequence is that for any estimator $\hat{\mathbf{X}} = \hat{\mathbf{X}}(\mathbf{A})$ of latent positions,

[3] we will assume all sets, measures and variables in/on $\mathbb{R}^d$ are Borel measurable

any orthogonally transformed estimator $\mathbf{X}' = \hat{\mathbf{X}} \cdot \mathbf{O}$ is the equivalent estimator of the identically distributed A', and there is in general no reason why two given estimators should share the same 'rotational orientation'. We call this phenomenon *orthogonal ambiguity of estimators*.

An important special case of RDPG are *stochastic block models*, which model scenarios where node connectivity is defined by the node belonging to a certain community. Here, we assume that the generating distribution $\mathfrak{F}$ is categorical, i.e. a mixture of atomic measures

$$\mathfrak{F} = \sum_{r \in [R]} \pi_r \cdot \delta_{\mathbf{c}_r} \tag{2}$$

for some $\mathbf{c}_1, \ldots, \mathbf{c}_R \in \mathbb{R}^d$ such that

$$\mathbf{B} := \operatorname{Gram}(\mathbf{c}_1, \ldots, \mathbf{c}_R) \in [0,1]^{R \times R} ,$$

and $\pi = (\pi_r)_{r \in [R]} \in [0,1]^d$ a probability vector (i.e. summing to one). In this particular RDPG, each $\mathbf{x}_i$ can be identified with some $\mathbf{c}_{\tau(i)}$, and we can write

$$p_{i,j} = \mathbf{c}_{\tau(i)}^\top \mathbf{c}_{\tau_j} = b_{\tau(i),\tau(j)} . \tag{3}$$

An alternative view is that

$$b_{k,l} = \mathbb{P}(\text{"Two given nodes, one in group } k\text{, one in group } l\text{, connect"}) .$$

We call $\mathbf{B}$ the block matrix and write in short $\mathbf{A} \sim \operatorname{SBM}(\pi, \mathbf{B})$. The mapping $\tau : [n] \to [R]$ is called the *community mapping*.

As for RDPG in general, the graph probability matrix $\mathbf{P}$ of an SBM built by the above procedure will be positive semidefinite, and each positive semidefinite $\mathbf{B}$ is the Gram matrix for some community vectors. One can more generally define SBM models with arbitrary block matrices $\mathbf{B}$; a positive semi-definite $\mathbf{B}$ models a *homophilic* network, where nodes in the same community are more likely to connect than nodes from different communities. We will here restrict ourselves to SBMs of the form (3), i.e. the positive semi-definite case.

Our main focus will be not on estimating $\mathbf{B}$, but the community mapping $i \mapsto \tau(i)$; estimating such community mappings will be our main focus in the results section.

Sometimes, SBMs are too restrictive, as they assume that all nodes within the same clique will have the same 'base frequency' for establishing nodes. The more general *degree–corrected SBM* (introduced in Karrer and Newman (2011)) replaces (3) with the alternative

$$p_{i,j} = w_i \cdot w_j \cdot \mathbf{c}_{\tau(i)}^\top \mathbf{c}_{\tau_j} \tag{4}$$

where $w = (w_i)_{i\in[n]}$ is a vector of weights in $[0,1]$. A simple generative model would assume a distribution $\mathfrak{F}$ on $[0,1] \times \mathcal{X}$, and $\mathbf{x}_i$ being modelled as $\mathbf{x}_i = w_i \cdot \mathbf{c}_{\tau(i)}$ with $\left(w_i, \mathbf{c}_{\tau(i)}\right) \sim \mathfrak{F}$. DCSBM models link an overall 'activity level' w_i of single nodes with community-based connectivity and are an attractive middle ground between the simplicity of SBM and the flexibility of a general RDPG.

Another extension of SBM models are *hierarchical SBM*, which assume a random assignment of nodes into a super-community which itself then is split into sub-communities. One particular straightforward model was introduced in Lyzinski et al. (2016), and can be formulated concisely as a latent embedding model where $\mathfrak{F}$ is a mixture

$$\mathfrak{F} = \sum_{k\in[K]} \pi_{(k)} \mathfrak{F}_{(k)} \tag{5}$$

where and $\left(\pi_{(k)}\right)_{k\in[K]}$ is a probability vector, and $\mathfrak{F}_k$ are discrete measures on $\mathbb{R}^d$ of the form (2), i.e.

$$\mathfrak{F}_{(k)} = \sum_{r\in[R_k]} \pi_{(k),r} \cdot \delta_{c_{(k),r}}$$

We unfold the clique vectors $\mathbf{c}_{k,r} \in \mathbb{R}^d$ into a matrix as

$$\mathbf{C} = \left[\mathbf{c}_{(1),1}, \ldots, \mathbf{c}_{(1),R_1}, \mathbf{c}_{(2),1}, \ldots, \mathbf{c}_{(K),R_K}\right]^\top$$

with dimension $n \times (\sum_k R_k)$, and collect the block probabilities as

$$\mathbf{\Pi} = \left[\pi_{(1),1}, \ldots, \pi_{(1),R_1}, \ldots, \pi_{(K),R_K}\right].$$

We write $\mathrm{HSBM}\left(K, \left(R_{(k)}\right)_{k\in[K]}, \pi, \mathbf{\Pi}, \mathbf{C}\mathbf{C}^\top\right) = \mathrm{RDPG}\left(\mathfrak{F}\right)$; in other words the HSBM model is built by first generating $\mathbf{X} = [\mathbf{x}_1| \cdots |\mathbf{x}_n]^\top$ iid according to $\mathfrak{F}$, defining $\mathbf{P} = \mathbf{X}\mathbf{X}^\top$ and generating $\mathbf{A} \sim \mathbf{P}$.

One easily sees that each hierarchical SBM is in fact an SBM with a particular structuring of its cliques. To justify the model, one hence needs to justify the two-step structuring of the cliques. The authors in Lyzinski et al. (2016) propose a strongly homophilic

structure. They compare the highest inter-subgraph connection probability

$$\text{max\_inter} := \max \left\{ \mathbf{c}_{(k),r}^{\top} \mathbf{c}_{(k'),r'} \ : \ k, k' \in [K], r \in [R_k], r' \in [R_{k'}], k \neq k' \right\}$$

with lowest intra-subgraph connection probability

$$\text{min\_intra} := \min \left\{ \mathbf{c}_{(k),r}^{\top} \mathbf{c}_{(k),r'} \ : \ k \in [K], r, r' \in [R_k] \right\}$$

and require that

$$\text{max\_inter} < \text{min\_intra} \, . \tag{6}$$

This makes the model *hierarchically homophilic*, i.e. connection likelihood between super-communities (subgraphs) is lower than within super-communities.

2.2 Estimators on random graphs

Our primary interest lies in community detection, and we cite relevant results for this in the context of SBM and HSBM models. This task has been approached in several ways; we refer to Abbe (2018) and Lee and Wilkinson (2019) for an overview. Popular methods for estimation are maximum likelihood, Bayesian frameworks, MCMC methods, spectral clustering and modularity maximization. We will focus on *spectral embedding* methods which are either applied to the observed adjacency matrix $\mathbf{A}$, or to the *normalized Laplacian*

$$\mathbf{L} := \mathbf{D}_{\mathbf{A}}^{-1/2} \mathbf{A} \mathbf{D}_{\mathbf{A}}^{-1/2}$$

with $\mathbf{D}_{\mathbf{A}}$ being the diagonal matrix of edge degrees (for node i, setting $\deg(i)$ as the number of edges containing i), i.e.

$$\mathbf{D}_{\mathbf{A}} = \operatorname{diag}\left(\deg(1), \ldots, \deg(n)\right) = \operatorname{diag}\left(\sum_{i \in [n]} a_{i,1}, \ldots, \sum_{i \in [n]} a_{i,n} \right) \, .$$

Since there are several variants of spectral embeddings defined and used in the literature, we will briefly define and relate the different variations. We recall the *singular value decomposition*; for a general $n \times m$ real matrix $\mathbf{M}$ we can find orthogonal matrices $\mathbf{U}(\mathbf{M}) \in \mathbb{R}^{n \times n}, \mathbf{V}(\mathbf{M}) \in \mathbb{R}^{m \times m}$ and a rectangular diagonal matrix $\mathbf{\Sigma}(\mathbf{M}) \in \mathbb{R}^{n \times m}$ with

non-negative entries such that

$$\mathbf{M} = \mathbf{U}(\mathbf{M})\mathbf{\Sigma}(\mathbf{M})\mathbf{V}(\mathbf{M})^\top .$$

If one assumes that the diagonal entries of $\mathbf{\Sigma}$ (the singular values) are in descending order, then $\mathbf{\Sigma}(\mathbf{M})$ is unique. If $\mathbf{M}$ is symmetric and positive definite, one can choose $\mathbf{U} = \mathbf{V}$.

In this setting, for $d \leq \min\{n, m\}$ we denote with $\mathbf{U}_d/\mathbf{V}_d$ the submatrix of $\mathbf{U}/\mathbf{V}$ containing the first d columns, and with $\mathbf{\Sigma}_d$ the $d \times d$ diagonal matrix containing the largest d singular values. Then the *rank d spectral approximation* of $\mathbf{M}$ is given by

$$\mathbf{M}_d := \mathbf{U}_d(\mathbf{M})\mathbf{\Sigma}_d(\mathbf{M})\mathbf{V}_d^\top(\mathbf{M}) .$$

We call

$$\hat{\mathbf{X}} = \hat{\mathbf{X}}_d(\mathbf{M}) := \mathbf{U}_d(\mathbf{M})\mathbf{\Sigma}_d^{1/2}(\mathbf{M}) \in \mathbb{R}^{m \times d}$$

a *(left hand) rank d spectral embedding*, and

$$\hat{\mathbf{Y}} = \hat{\mathbf{Y}}_d(\mathbf{M}) := \mathbf{V}_d(\mathbf{M})\mathbf{\Sigma}_d^{1/2}(\mathbf{M}) \in \mathbb{R}^{m \times d}$$

a *(right hand) rank d spectral embedding* of $\mathbf{M}$.

If $\mathbf{M}$ is symmetric and positive semidefinite, one can achieve $\mathbf{U}_d = \mathbf{V}_d$, and in this case $\hat{\mathbf{X}}_d(\mathbf{M}) = \hat{\mathbf{Y}}_d(\mathbf{M})$. Further, in this case the spectral embedding gives rise to an approximate factorization of $\mathbf{M}$, since

$$\hat{\mathbf{X}}\hat{\mathbf{X}}^\top = \mathbf{V}_d(\mathbf{M})\mathbf{\Sigma}_d(\mathbf{M})\mathbf{V}_d^\top(M) = \mathbf{M}_d .$$

This is in particular true when choosing[4] $\mathbf{M} = \mathbf{P}$ as the spectral embedding of graph probability matrices generated by RDPG.

If $\mathbf{M}$ is symmetric but not necessarily positive semidefinite, one can instead consider the eigendecomposition of $\mathbf{M}$ as

$$\mathbf{M} = \mathbf{Q}(\mathbf{M})\mathbf{\Lambda}(\mathbf{M})\mathbf{Q}(\mathbf{M})^\top$$

[4]This type of embedding is often denoted as $\tilde{\mathbf{X}}$ or simply $\mathbf{X}$. We will use it only very sparingly and will in these cases write out $\hat{\mathbf{X}}_d(\mathbf{P})$ to avoid confusion and additional notational burden.

with $\mathbf{\Lambda}(\mathbf{M})$ containing the eigenvalues of $\mathbf{M}$, decreasing in order of absolute value, and $\mathbf{Q}(\mathbf{M})$ a corresponding orthogonal eigenvector matrix. With restriction to the first d rows/columns as before, we can then set

$$\widetilde{\mathbf{M}}_d := \mathbf{Q}_d(\mathbf{M})\mathbf{\Lambda}_d(\mathbf{M})\mathbf{Q}_d(\mathbf{M})^\top \tag{7}$$

as order d eigenapproximation of $\mathbf{M}$, and

$$\widetilde{\mathbf{X}} = \widetilde{\mathbf{X}}_d(\mathbf{M}) := \mathbf{Q}_d(\mathbf{M})|\mathbf{\Lambda}|^{1/2}$$

as a *symmetric rank d spectral embedding* of $\mathbf{M}$. From the eigendecomposition $\mathbf{M} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$, one readily sees that with either

- $\mathbf{U} = \mathbf{Q}$, $\mathbf{V} = \mathbf{Q} \cdot \operatorname{diag}(\operatorname{sign}(\lambda_i))$, or
- $\mathbf{V} = \mathbf{Q}$, $\mathbf{U} = \mathbf{Q} \cdot \operatorname{diag}(\operatorname{sign}(\lambda_i))$

one can achieve a SVD of the form $\mathbf{M} = \mathbf{U}\,|\mathbf{\Lambda}|\,\mathbf{V}^T$. Thus, for a symmetric matrix $\mathbf{M}$, one can always construct left and right spectral embeddings as variants of a symmetric spectral embedding.

The literature meanders between using one or the other variant, but for symmetric matrices like adjacency or Laplacian matrices, the differences lie in the sign of some eigenvectors, and moreover all approaches are equivalent for positive semi definite matrices like graph probability matrices. We will use the *right hand* spectral embedding as 'the' spectral embedding, as this corresponds to the setup used later in unfolded spectral adjacency/Laplacian embeddings.

We now establish an intuitive reasoning for why spectral embeddings are natural candidates for a good way to approximate $\mathbf{X}$. From the above we infer that for a graph probability matrix $\mathbf{P}$, its spectral embeddings lead to an approximate low rank d factorization

$$\mathbf{P} \approx \mathbf{P}_d = \hat{\mathbf{X}}_d(\mathbf{P}) \cdot \hat{\mathbf{X}}_d(\mathbf{P})^\top \,.$$

In an RPDG setting, $\mathbf{P}$ is generated by a rank d factorization $\mathbf{P} = \mathbf{X}\mathbf{X}^\top$, and hence one can hope $\hat{\mathbf{X}}_d(\mathbf{P})$ to be a meaningful approximation for $\mathbf{X}$ (up to orthogonal ambiguity). We do not have direct access to $\mathbf{P}$ or $\hat{\mathbf{X}}_d(\mathbf{P})$; however we can calculate $\hat{\mathbf{X}}_d(\mathbf{A})$ for $\mathbf{A} \sim \mathbf{P}$ and can hope that, conditional on $\mathbf{P}$, this is (in a meaningful sense) a consistent estimator for $\hat{\mathbf{X}}_d(\mathbf{P})$.

We will formulate one asymptotic result rigorously. Fix d and assume that we have an infinite sequences $\mathbf{z}_i$ drawn i.i.d. from a d-dimensional distribution $\mathfrak{F}$. For $n \in \mathbb{N}$ and $i \in [n]$ we set $\mathbf{x}_i^{\{n\}} = \rho_n^{1/2}\mathbf{z}_i$ for some deterministic sequence $\rho_n \in [0,1]$. (ρ_n is a global scaling of the average node degree.) Let $\mathbf{A}^{\{n\}}$ be the sequence of $n \times n$ adjacency matrices generated from $\mathbf{X}^{\{n\}}$ via (1), and define $\hat{\mathbf{X}}^{\{n\}} = \hat{\mathbf{X}}_d(\mathbf{A}^{\{n\}})$, with row vectors $\hat{\mathbf{x}}_i^{\{n\}}$, $i \leq n$. The following result is a slightly weaker variant of (Rubin-Delanchy et al., 2022, Thms. 1, 2):

Theorem 2.1. There is a universal constant $k > 0$ such that that the following holds: Assume that $\rho_n \geq cn^{-1/2}(\log n)^k$ for some $c > 0$ and all n, and denote with $\|\cdot\|_2$ the Euclidean norm in $\mathbb{R}^d$. Then for suitable orthogonal matrices $\mathbf{Q}^{\{n\}} \in \mathbb{R}^{n\times n}$ the normalized error sequence

$$\frac{\max_{i\in[n]} \left\| \mathbf{Q}^{\{n\}}\hat{\mathbf{x}}_i^{\{n\}} - \mathbf{x}_i^{\{n\}} \right\|_2}{n^{-1/2}(\log n)^{k/4}}$$

is almost surely bounded from above[5], and moreover for a fixed m and $n \geq m$, the m–dimensional error vectors

$$\sqrt{n} \cdot \left(\left\| \mathbf{Q}^{\{n\}}\hat{\mathbf{x}}_i^{\{n\}} - \mathbf{x}_i^{\{n\}} \right\| \right)_{i\in[m]}$$

converge in distribution to some multivariate normal distribution $\mathcal{N}(0, \mathbf{\Gamma}(\mathbf{X}))$.

Intuitively, this result asserts that, if the node degree decays slower that $n^{-1/2}$, $\hat{\mathbf{X}}$ is (up to orthogonal ambiguity) a consistent estimator for $\mathbf{X}$ and even obeys a central limit theorem.

Following the discussion in (Rubin-Delanchy et al., 2022, Section 4.1), consider the special case of SBM with a fixed community size, i.e. $\mathbf{z}_i = \mathbf{c}_{\tau(i)}$ with random community assignment τ. Then this theorem entails that, if ρ_n decays slower than $n^{-1/2}$, $\hat{\mathbf{X}}_i^{\{n\}}$ will, up to orthogonal ambiguity, be approximately normally distributed around $\rho_n^{1/2}\mathbf{c}_{\tau(i)}$, i.e. it will behave in limit like a Gaussian Mixture model.

Interpreting this result for a DCSBM, $\hat{\mathbf{X}}$ should be close to $\rho_n^{1/2} w_i \mathbf{c}_{\tau(i)}$. If one is interested in community classification, then the additional individual 'node intensity' w_i is not relevant and even can distort clustering (especially for smaller values of w_i).

[5]The authors prove a stronger result using the $O_{\mathbb{P}}$ notation. The notation $g = O_{\mathbb{P}}(f)$ for random sequences g, f means that the probability for the event $g(n) > Cf(n)$ decays faster than any polynomial when $n \to \infty$. However, a simple application of the Borell-Cantelli Lemma shows that if $g = O_{\mathbb{P}}(f)$, then g/f is almost surely bounded.

Sanna Passino et al. (2022) proposes to use spherical coordinates of the rows of $\hat{\mathbf{X}}$ instead, which project $\mathbb{R}^d$ onto its polar coordinates $[0, 2\pi)^{d-1}$. More explicitly, a vector $\mathbf{x} = (x_1, \dots, x_d)$ is mapped to $\theta = f_d(\mathbf{x})$, where the coordinates of θ are given by

$$\theta_j = \begin{cases} \arccos(x_2/\|x_{:2}\|) & j = 1, x_1 \geq 0 \ , \\ 2\pi - \arccos(x_2/\|x_{:2}\|) & j = 1, x_1 < 0 \ , \\ 2\arccos(x_{j+1}/\|x_{:j+1}\|) & j > 2 \ . \end{cases} \tag{8}$$

This transformation is continuous except for the coordinate hyper-planes. If we assume that $\mathbf{x}_i$ and $\hat{\mathbf{x}}_i$ have zero probability of hitting these hyper-planes, and denote the transformed vectors as $\hat{\theta}_i = f_d(\hat{\mathbf{x}}_i)$, then $\hat{\mathbf{\Theta}} := [\hat{\theta}_1 | \cdots, |\hat{\theta}_n]^\top$ is a normalized version of $\hat{\mathbf{X}}$, and under the assumptions of Theorem 2.1, $\hat{\mathbf{\Theta}}$ is an asymptotically consistent estimator for $\mathbf{\Theta} := [f_d(\mathbf{x}_1) | \cdots | f_d(\mathbf{x}_n)]^\top$. We will in this setting call $\hat{\mathbf{\Theta}}$ the *normalized ASE*.

We now turn to the setting of HSMB. Here, Lyzinski et al. (2016) establishes a procedure for estimating super-community and sub-community membership as well as motif equality for communities. The proposed approach is a two-step process in which the graph is first divided into subgraphs/super-communities based on a modified nearest neighbour clustering algorithm 1.

Algorithm 1: Seeded nearest neighbour subspace clustering

Data: Estimated embedding $\hat{\mathbf{X}} \in \mathbb{R}^{n \times d}$, number of communities K
Result: Estimated community membership $\hat{\tau} : [n] \to [K]$
Set $\mathcal{S}$ as random sampling of K distinct rows from $\hat{\mathbf{X}}$;
for $i \in [n]$ **do**
  Find $\mathbf{y}, \mathbf{z}$ in S_{i-1} with maximal similarity $\mathbf{y}^\top \cdot \mathbf{z}$;
  If $\hat{\mathbf{x}}_i^\top \cdot \mathbf{s} \leq \mathbf{y}^\top \cdot \mathbf{z}$ for all $\mathbf{s} \in \mathcal{S}_{i-1}$, set $\mathcal{S}_i = (\mathcal{S}_{i-1} - \{\mathbf{z}\}) \cup \{\mathbf{x}_i\}$;
Label $\mathcal{S}_n = \{\mathbf{s}_1, \dots, \mathbf{s}_K\}$;
for $i \in [n]$ **do**
  Assign $\hat{\tau}(i) = \text{argmax}_j \ \hat{\mathbf{x}}_i{}^\top \mathbf{s}_j$;

This algorithm starts with a random seed of centers, and tries n times to replace one of the two closest points with the ith point, if that point is less similar to/farther from all currently selected points. It is used in the main algorithm (Lyzinski et al., 2016, Algorithm 1), which is concerned with identification of super-communities, motif identification and estimation of block structure.

This algorithm is efficient, if we consider classification error up to re-labeling. Let $\mathcal{P}_R$ denote the set of permutations (bijections) $\sigma : [R] \to [R]$, and for two classifications

$\tau, \hat{\tau} : [n] \to [R]$, set the error count

$$\mathrm{Err}(\hat{\tau}, \tau) := \min_{\sigma \in \mathcal{P}_R} \sum_{i \leq n} \mathbf{1}[\tau(i)! = \sigma(\hat{\tau(i)})] \ .$$

This definition asks 'Given the optimal re-labeling of clusters, how many nodes are labeled incorrectly by $\hat{\tau}$'? Part of the main result (Lyzinski et al., 2016, Lemma 6) can be formulated as below:

Theorem 2.2. Assume a HSBM model of the form (5) with fixed and known $d, D, K, \left(R_{(k)}\right)_{k \in [K]}$. Assume further that (6) holds and that $\min_i \pi(i) > 0$. Further assume an i.i.d. series of embeddings $(\mathbf{x}_i)_{i \in \mathbb{N}}$ drawn according to $\mathfrak{F}$ with corresponding super-community assignment $\tau_{sup}(i) = k$ iff $\mathbf{x}_i \in \left\{\mathbf{c}_{(k),1}, \ldots, \mathbf{c}_{(k),R_{(k)}}\right\}$, and consider a series of adjacency matrices $\mathbf{A}^{\{n\}} \in \mathbb{R}^{n \times n}$ generated from $(\mathbf{x}_i)_{i \in [n]}$ respectively. For each n let $\hat{\tau}^{\{n\}}_{sup} : [n] \to [K]$ be the corresponding community estimators from algorithm 1. Then almost surely $\mathrm{Err}(\hat{\tau}, \tau) \to 0$.

The authors apply this clustering algorithm iteratively (interspersed with an algorithm for detecting motifs) in their main algorithm, which iteratively identifies hierarchies of communities. However, if we consider two-level hierarchies with SBM or DCSBM assumptions in the sub communities, it is natural to combine the super-community clustering with a more traditional sub-community clustering algorithm, like in algorithm 2:

First this algorithm considers a (potentially high-dimensional) embedding and identifies a cluster in there. Following this first clustering, each identified cluster (super-community) is then considered as a network of its own, and clustered into sub-communities according to standard methodology, based on ASE restricted to this sub-graph (and using a potentially lower embedding dimension than the global embedding).

The algorithms above assume knowledge of $D, d, K, (R_k)$ which in practice is not a given. Lyzinski et al. (2016) suggests using singular value thresholding for estimating D, and establishes a tailored method for estimating K (algorithm 3).

In words, the algorithms returns a Monte Carlo estimate for the largest similarity in our optimized seed for clustering. If k is smaller than the true cluster size, one would expect ϕ_k to be small as there is enough opportunity to space out k points (assuming

Algorithm 2: Super community and community detection in HSBM

Data: Adjacency matrix $\mathbf{A} \in \{0,1\}^n \times n$, $D, d, K, (R_k)_{k \leq K}$
Output: Estimator $\hat{\tau}(i) = (k, r)$ for membership in super-community k and sub-community ν

Calculate D–dimensional ASE $\hat{\mathbf{X}} = \hat{\mathbf{X}}_D(\mathbf{A})$;

Use clustering algorithm 1 with input $\hat{\mathbf{X}}, K$ for a super–community estimator $\hat{\tau}_{sup} : [n] \to [K]$;

for $k \in [K]$ **do**

- Define $J_{(k)} := \hat{\tau}_{sup}^{-1}(\{k\})$, induced subgraphs $\hat{H}_{(k)} := \mathrm{Sub}(G, J_k)$ and calculate adjacency matrices $\mathbf{A}_{(k)} = (a_{i,j})_{i,j \in J_{(k)}}$ of the subgraphs;
- Calculate d dimensional ASE $\hat{\mathbf{X}}_{(k)} = \hat{\mathbf{X}}_d(\mathbf{A}_{(k)})$;
- Estimate community assignment $\hat{\tau}_{(k)}^{sub}(i) : J_k \to [R_k]$ from clustering $\hat{\mathbf{X}}_{(k)}$ using GMM ;

Combine to a joint estimator

$$\hat{\tau}(i) := \sum_{k \in [K]} \mathbf{1}[\hat{\tau}_{sup}(i) = k] \cdot \hat{\tau}_{(k)}^{sub}(i) \ .$$

Algorithm 3: K estimation in HSBM

Data: Adjacency matrix $\mathbf{A} \in \{0,1\}^n \times n$, D, N
Output: Estimated cluster size $\hat{K}$

Estimate $\hat{X} = \hat{X}_D(\mathbf{A})$ from ASE;

for $k \in [D]$ **do**

- **for** $i \in [N]$ **do**
  - Run algorithm 1 on $\hat{X}$ with k clusters, returning set of clustering centers $\mathcal{S}(i, f)$;
  - Set $\phi_{i,k} = \max\left\{\mathbf{s}^\top \mathbf{t} : \ \mathbf{s}, \mathbf{t} \in \mathcal{S}(i,k), \mathbf{s} \neq \mathbf{t}\right\}$;
- Set $\phi_k = \sum_{i \in [N]} \phi_{i,k}/N$;

Use the elbow algorithm to find $\hat{K}$ from scores $\phi_1, \ldots \phi_D$.

the separation of clusters is strong enough). If k is larger however, two points must lie in the same true cluster and (assuming clusters are fairly tight) ϕ^k should be fairly large.

A second question of practical relevance is how to handle DCSMB models, in particular since heterogeneous node intensities could lead to mis-classifications of the super-communities/subgraphs. (An HDCSBM is completely analogue to the HSMB as above, but instead of SBM models, the $\mathfrak{F}_k$ map DCSMB models on their respective subgraphs. We will define a generalization of this concept further below and will not write this model out.) It is straightforward to replace the usage of ASE in both steps of algorithm 2, with normalized ASE estimates, which also translates to the estimation of R. Since we will introduce a further generalization in the next section, we postpone writing out an algorithm until then.

2.3 Dynamic networks

After discussing random graphs we can finally turn to our main focus, dynamic networks. We define a *dynamic network* to be a time series of random graphs. We will focus on time-discrete networks over a fixed horizon, i.e. a series[6] $(G^t)_{t\in[T]}$ of randomly generated graphs. We will further assume that the node sets are fixed and equal to $[n]$, and hence all relevant information is found in the $n \times n$ adjacency matrices $(\mathbf{A}^t)_{t\in[T]}$.

In practice, one often observes edges registering at random times, i.e. one would observe a time indexed random point process $(\tau_l, x_l, y_l)_{l\in[L]}$ of random times τ_l and random node indices x_l, y_l, noticing that at time τ_l, a connection between nodes x_l and y_l was registered. This is an alternative definition for dynamic networks, and if we restrict ourselves to bounded τ_l, we can translate this into a dynamic network as described above, by first choosing a time resolution T, then uniformly scaling τ_l to times $\theta_l = T \cdot (\tau_l - \min_m \tau_m)/(\max_m \tau_m - \min_m \tau_m)$, and then setting

$$\mathbf{A}^t_{i,j} := \begin{cases} 1, \text{ if } \exists l \in [L] \text{ s.t. } \theta_l \in (t-1, t], (x_l, y_l) \in \{(i,j),(j,i)\} \\ 0, \text{ otherwise.} \end{cases},$$

In words, G^t shows an edge between i and j iff at some random (normalized) time between the (normalized) times $t-1, t$, a connection was registered. This is a lossy

[6] Recall that the notation G^t, $\mathbf{A}^t$, as described in the section on notation, does not mean the usual power of e.g. a matrix but is simply an indexing for the time dimension.

transformation as we lose the multiplicity of connections and the exact observation times, and the choice of T matters - too coarse and too much information is lost, too fine and the graphs will be almost completely disconnected.

Expanding the RDPG generative model into a dynamic version has been achieved using different approaches, see Sanna Passino et al. (2021) for an overview. We note as one prominent model that of COSIE, which essentially models the probability matrix $\mathbf{P}^t$ as stemming from a static embedding of nodes, but introducing a time-varying matrix in between, that is

$$\mathbf{P}^t = \mathbf{X}\mathbf{\Lambda}^t\mathbf{X}^\top .$$

Another natural extension is that of a *dynamic latent embedding*, which assumes a dynamic embedding (introduced in Sarkar and Moore (2005), a review of results is found in Kim et al. (2018)). Here one assumes an i.i.d. series of paths $\mathbf{z}_i = (\mathbf{z}_i^t)_{t\in T}$ drawn from some distribution $\mathfrak{F}$ on $\mathbb{R}^{d\times T}$, which generate probability matrices as

$$\mathbf{P}^t = \left(\kappa(\mathbf{z}_i^t, \mathbf{z}_j^t\right)_{i,j\in[n]}, \qquad t \in [T] , \tag{9}$$

and then for each t generate $\mathbf{A}^t \sim \mathbf{P}^t$, independently given $(\mathbf{P}^t)_{t\in[T]}$.

The *multilayer random dot product graph* model established in Jones and Rubin-Delanchy (2020) generalizes both notions. We need to establish a couple of assumptions and notations. (We simplify the original definition slightly to ease notation.)
We recall from the notations section that given some indexed sequence of matrices $(\mathbf{A}_j)_{j\in[J]}$ of dimensions $n \times m_j$, then the *unfolding* of the matrices $\mathbf{A} = [\mathbf{A}_1|\cdots|\mathbf{A}_J]$ is the $n \times \sum_{j\in[J]} m_j$ matrix obtained by writing the matrices side by side.

Given some $n, T \in \mathbb{N}$ and a fixed dimension $d \in \mathbb{N}$, we assume a sequence of dimensions $d^t \in \mathbb{N}$ for $t \in [T]$, a sequence of fixed $n \times d^t$ matrices $\mathbf{\Lambda}^t$, and subspaces $\mathcal{X} \subseteq \mathbb{R}^d$ and $\mathcal{Y}^t \subseteq \mathbb{R}^{d^t}$ such that, for all t and all $\mathbf{x} \in \mathcal{X}, \mathbf{y}^t \in \mathcal{Y}^t$ we have $\mathbf{x}^\top\mathbf{\Lambda}^t\mathbf{y}^t \in [0, 1]$. We further assume that we have a distribution $\mathfrak{F}$ on $\mathcal{X} \times (\mathcal{Y}^1) \times \cdots \times \left(\mathcal{Y}^T\right)$, and let matrices $\mathbf{X}, \mathbf{Y}^{(1)}, \ldots, \mathbf{Y}^{(T)}$ be i.i.d. samples drawn from $\mathfrak{F}$ (i.e. the i-th row of all matrices contains the result of the i–th independent sample). We unfold $\mathbf{Y}^t$ and $\mathbf{\Lambda}^t$ into the matrices

$$\mathbf{\Lambda} = \left[\mathbf{\Lambda}^1|\cdots|\mathbf{\Lambda}^T\right] \in \mathbb{R}^{d\times\sum_t d^t}, \quad \mathbf{Y} = \left[\mathbf{Y}^1|\cdots|\mathbf{Y}^T\right] \in \mathbb{R}^{n\times\sum_t d^t},$$

and define

$$\left[\mathbf{P}^1|\cdots|\mathbf{P}^T\right] = \mathbf{P} := \mathbf{X}\mathbf{\Lambda}\mathbf{Y}^\top \,. \tag{10}$$

If we then generate adjacency matrices $\mathbf{A}^t \sim \mathbf{P}^t$ as a time series (independent given $\mathbf{P}$), and unfold $\mathbf{A} = \left[\mathbf{A}^1|\cdots|\mathbf{A}^T\right] \in \mathbb{R}^{n\times Tn}$, we call $\mathbf{A}$ the adjacency matrices of a multi layer random dot product graph and write $A \sim \text{MRDPG}\left(\mathfrak{F}, \mathbf{\Lambda}\right)$.

(The original definition also makes some further technical specifications meant to distinguish subclasses, and allows for varying n over time. We do not need these and hence omit them.)

The MRDPG setup might not seem to fit our intended goal of describing dynamic networks - it prescribes a setting in which closeness of one fixed embedding $\mathbf{X}$ to a time series $\mathbf{Y}^t$ of embeddings, transformed deterministically via $\mathbf{\Lambda}^t$, determines edge probabilities. However, in Theorem 1 of Gallagher et al. (2021), it is proven that a dynamic latent embedding model of the form (9), where κ admits a finite eigendecomposition, can be re-written as an MRDPG model of the form (10), where each row vector $\mathbf{x}_i$ of $\mathbf{X}$ can be computed from $\mathbf{z}_i$, and for each t, each row vector $\mathbf{y}_i^t$ can be computed from $\mathbf{z}_i^t$. One can interpret $\mathbf{X}$ as a time-independent component of the embedding of nodes, while the $\mathbf{Y}^t$ capture a time varying component.

We explore now the question of embeddings and estimators for dynamic models estimation. The simplest adaptation of ASE or LSE to the dynamic case would be to apply them individually to each t. However, as was noted in Sanna Passino et al. (2021), orthogonal ambiguity in general leads to embedding estimators $\widehat{X}^t$ that are not orthogonally aligned, which then requires the application of Procrastes procedures to calculate orthogonally aligned versions of the estimators. The MRDPG approach yielded a novel embedding method, called *unfolded ASE* (citing the version from Gallagher et al. (2021):

Algorithm 4: UASE

Data: Matrices $\mathbf{M}^1, \ldots, \mathbf{M}^T$, embedding dimension d

Result: Orthogonally aligned node embeddings $\hat{\mathbf{Y}}^1, \ldots, \hat{\mathbf{Y}}^T$

Set $\mathbf{M} = \left[\mathbf{M}^1|\cdots|\mathbf{M}^T\right]$;

Find $\mathbf{M}_d = \mathbf{U}_d\mathbf{\Sigma}_d\mathbf{V}_d^T$ as rank d singular value approximation of $\mathbf{M}$;

Define $\hat{\mathbf{Y}} = \mathbf{V}_d\mathbf{\Sigma}_d^{1/2}$;

Split $\hat{\mathbf{Y}} = \left[\hat{\mathbf{Y}}^1|\cdots|\hat{\mathbf{Y}}^T\right]$;

(Recalling the previous definitions, $\hat{\mathbf{Y}}$ is the rank d right hand adjacency spectral embedding of the $n \times Tn$ matrix $\mathbf{M}$.)

This algorithm wiill normally be applied to adjacency matrices $\mathbf{A}^t$. It was shown (Gallagher et al., 2021, Propositions 2,3) that these estimators $\hat{\mathbf{Y}}_d(\mathbf{A})$ are consistent in the sense that they converge to the right hand adjacency spectral embedding $\hat{\mathbf{Y}}_d(\mathbf{P})$ (called noise–free embedding there and denoted $\widetilde{\mathbf{Y}}$), which in turn can be written as linear combinations of the $\mathbf{y}_i^t$. This does not necessarily yield a consistent estimator of the underlying dynamic latent embedding $\mathbf{z}_i^t$, but as a guiding intuition it should capture the random evolution part of its generating model. Further, it is reasonable to expect that, for a community setting like SBM, the estimators $\hat{\mathbf{Y}}_d(\mathbf{P})$ should tend to cluster according to community membership.

Formal properties of cross-sectional and longitudinal stability of $\hat{Y} = \hat{Y}_d(\mathbf{A})$ w ere proven which sharpen this intuition: As a special case of (Gallagher et al., 2021, Corollary 5), we have that

- If, for some fixed t, we have $\mathbf{z}_i^t = \mathbf{z}_{i'}^t$ for some $i, i' \in [n]$, then the corresponding row vectors $\hat{\mathbf{y}}_i^t, \hat{\mathbf{y}}_{i'}^t$ of $\mathbf{Y}^t$ are asymptotically equal;
- If the probability matrices $\mathbf{P}^t$ and $\mathbf{P}^{t'}$ are almost surely equal for some t, t', then $\hat{\mathbf{Y}}^t$, $\hat{\mathbf{Y}}^{t'}$ are almost surely asymptotically equal.

The first property intuitively implies that, if we use some dynamic SBM model, clustering the rows of $\hat{\mathbf{Y}}^t$ should lead to asymptotically correct community detection for any fixed time point t. The second property assures us that in the static case where $\mathbf{Z}^{t+1} = \mathbf{Z}^t$, the estimator $\hat{\mathbf{Y}}^t$ should also be asymptotically static. Furthermore, the single estimators $\hat{\mathbf{Y}}^t$ are naturally orthogonally aligned, which is a strong advantage over individually estimated ASE.

The authors then informally introduce an algorithm (during the development of the real data example in section 5) for community detection in a dynamic setting. Implicitly, they extend the setup from a dynamic SBM to a dynamic degree-corrected SBM.

Algorithm 5: UASE based community detection in dynamic DCSBM

Data: Adjacency matrices $\mathbf{A}^1, \ldots, \mathbf{A}^T$
Result: Estimated embedding dimension $\hat{d}$, estimates number $\hat{R}$ of blocks, estimated block membership $\hat{\tau}(i,t)$ for node i and time t
Estimate $\hat{d}$ using profile likelihood;
Apply algorithm 4 to obtain $\hat{\mathbf{Y}} = \left[\hat{\mathbf{Y}}^1 | \cdots | \hat{\mathbf{Y}}^T\right]$;
Transform $\hat{\mathbf{Y}}$ into spherical coordinates $\hat{\mathbf{\Theta}} = \left[\hat{\mathbf{\Theta}}^1 | \cdots | \hat{\mathbf{\Theta}}^T\right]$ following (8);
Fit a Gaussian mixture model with varying covariances to the union of points $\mathcal{P} = \bigcup_t \hat{\mathbf{\Theta}}^t$ in $\mathbb{R}^d$, over a range of candidate cluster sizes $1, \ldots, R$;
Choose a cluster size $\hat{R}$ for $\mathcal{P}$ from Bayesian Information Criterion;
Cluster $\mathcal{P}$ using the maximal a posteriori Gaussian cluster membership, and return the estimator as a dynamic community membership estimator $\hat{\tau}(i,t)$.

3 Model

Our contribution is two-fold. In this section, we establish a new hierarchical model for dynamic networks which generalizes both dynamic SBM and static hierarchical SBM models introduced above. Further, we establish two methodologies for estimation of super- and sub-community membership in this dynamic hierarchical model, which are inspired by the presented algorithms for static hierarchical classification as well as the UASE approach for dynamic DCSBMs. The main method also contains a dimensional selection from UASE based on temporal stability as a novel element.

In the next section, we apply our estimators to both simulated data and a real-world data set, and report on the performance and observed patterns.

3.1 The dynamic hierarchical model

We consider a dynamic generalization of the HSBM model, using the ideas and techniques developed for dynamic SBMs. Namely, assume we are given

- $n, d, K, T \in \mathbb{N}$, and a static probability vector $\Pi = \left(\pi_{(k)}\right)_{k\in[K]}$ of length K,
- for each $k \in [K]$, a block size $R_{(k)}$ and a set of community vectors

$$\mathcal{C}_{(k)} = \left\{\mathbf{c}_{(k),1}, \ldots, \mathbf{c}_{(k),R_k}\right\} \subseteq \mathbb{R}^d ,$$

- for each $k \in [K]$, a distribution $\mathfrak{F}_{(k)}$ on $\left([0,1] \times \mathcal{C}_{(k)}\right)^T$ generating paths of random pairs of weights in $[0,1]$ and community centers from $\mathcal{C}_{(k)}$.

Given these assumptions, we consider the mixture distribution

$$\mathfrak{F} = \sum_{k \in [K]} \mathfrak{F}_k \ .$$

We assume that for each node i, we draw independently a path $((w_i^t, \mathbf{x}_i^t))_{t \in [T]}$ according to $\mathfrak{F}$ (i.e. a path from $\mathfrak{F}_k$ with a randomly chosen k), and define the latent embedding as $\mathbf{z}_i^t = w_i^t \cdot \mathbf{x}_i^t$. Finally we draw $\mathbf{A}^t \sim \mathbf{P}^t$ with $\mathbf{P}^t = \mathbf{Z}^t \left(\mathbf{Z}^t\right)^\top$.

We call this model a *dynamic hierarchical degree corrected SBM (DHDCSBM).*

This model blends static and dynamic elements. Each node will be randomly but statically assigned to one super-community carrying a DCSBM model, and each DCSBM model has a static and deterministic set of community vectors. Membership in the sub-community within the given super-community and node intensity however can evolve dynamically over time. In a compact notion, one can express the community assignment as a random mapping $\tau(n,t) = (k,r)$ with $k \in [K]$ being the index of the super-community and being constant over time, and $r \in [R_k]$ being the index of the sub–community and potentially varying over time. Our main objective is to develop a method to estimate *super-community and sub-community membership* $\tau(i,t) = (k,r)$ from $\mathbf{A} = \left[\mathbf{A}^1 | \cdots | \mathbf{A}^T\right]$.

3.2 Simulated data

In the github repository Creutzig (2025), we provide a method to generate a simulated DHDCSBM, called `make_hierarchical_model`. This method implements a specific form of the general model, by assuming first that the values of $\mathbf{P}^t$ (and hence the scalar products of our community vectors) should be restricted to

$$p_{i,j}^t = \begin{cases} 0, & i = j, \\ \rho_1, & i,j \text{ in different super-communities}, \\ \rho_2, & i,j \text{ in same super- but different sub-communities}, \\ \rho_3, & i,j \text{ in same sub-community}. \end{cases}$$

Our implementation requires $\rho_1 \leq \rho_2 \leq \rho_3$. Further, conditional on node i being assigned to super-community k with R_k sub-communities, the random process $s_1, \dots s_T$ of sub-community assignment for node i is assumed to follow a simple Markov process of the form:

- We draw $s_1 \in [R_{(k)}]$ according to a given probability vector $\pi_{(k)} \in [0,1]^{R_{(k)}}$.
- Conditional on $s_t = s$, with probability p_{stay} we set $s_{t+1} = s$, and with probability $1 - p_{stay}$ we draw s_{t+1} independent from prior history and according to $\pi_{(k)}$.

Further, the model will simulate the dynamic weights $w_{i,t}$ of the dynamic DCSBM model as drawn i.i.d. over nodes and time from a Beta distribution with super-community specific parameters $\alpha_{(k)}, \beta_{(k)}$.

The method `make_hierarchical_model` accepts the following parameters:

- The total size n of the graph,
- The number T of time intervals,
- Connection probabilities $\rho_1 \leq \rho_2 \leq \rho_3$,
- A vector `R` of sub-community sizes, with elements R_k and length K,
- A super community assignment probability vector `sup_probs` of length K,
- A list `sub_probs` of length K, containing sub community assignment probability vectors $\pi_{(k)}$ of length R_k,
- The value `stay_prob` $\in [0,1]$,
- Arrays `alpha, beta` of length K, for the Beta parameters of the node weights.

The method will then

1. Calculate some[7] embedding dimension d and generate d–dimensional sub-community vectors $\mathbf{c}_{(1),1}, \dots \mathbf{c}_{(1),R_1}, \dots, \mathbf{c}_{(K),R_K}$ such that

$$\mathbf{c}_{(k),r}^{\top} \mathbf{c}_{(k'),r'} = \begin{cases} \rho_1, & k \neq k' \ , \\ \rho_2, & k = k', r \neq r' \ , \\ \rho_3, & k = k', r = r' \ . \end{cases}$$

[7] For convenience, the embedding dimension used is chosen simply rather than optimally.

2. Randomly dissect the nodes into subgraphs $H_{(1)}, \ldots, H_{(K)}$, according to `sup_probs` ,
3. For node i assigned to subgraph k, build a Markov process $s_1, \ldots s_T \in [R_{(k)}]$ of assignments to sub-communities $R_{(k)}$ as described above (using `sub_probs` for the assignment), and generate an i.i.d. sequence $w_{i,t}$ of Beta$(\alpha_{(k)}, \beta_{(k)})$ distributed weights;
4. Set $\mathbf{z}_i^t = w_{i,t} \cdot \mathbf{c}_{(k),s_t}$, $\mathbf{Z}^t = [\mathbf{z}_1^t | \cdots | \mathbf{z}_n^t]^\top$, and finally $\mathbf{Z} = \left[\mathbf{Z}^1 | \cdots | \mathbf{Z}^T\right]$.
5. Calculate $\mathbf{P}^t = \mathbf{Z}^t \left(\mathbf{Z}^t\right)^\top$ and simulate for each t independently $\mathbf{A}^t \sim \mathbf{P}^t$.

We note several specializations of the generating model:

- For $T = 1$ this generates a hierarchical DCSBM random graph;
- For $K = 1$ this generates a dynamic DCSBM model;
- For $\alpha = 100, \beta = 1$ the individual node weights are almost constant 1 and we end up with a dynamic hierarchical SBM model;
- For $\rho_1 = \rho_2$, the model is equivalent to a non-hierarchical dynamic DCSBM;
- For $p_{stay} = 1$, the model becomes a static hierarchical DCSBM random graph.

A visualization of simulated data is shown in the dashboard 1, showing the result of a simulation run with default parameters (column "Default" in Table 1).

1. On top left, the super- and sub-community distribution of nodes at $t = 0$, as a sunburst chart;
2. On top right, the size (node count) of sub communities over time;
3. On middle left, the relative edge density (number of actual connections, divided by number of possible connections) between nodes, aggregated over all time intervals and grouped by super-community pairs;
4. On middle right, the relative edge density (number of actual edges divided by number of possible edges), grouped by edge type (within sub-communities, between sub-communities but within super-communities, and between super-communities), grouped by time step;
5. On bottom left, a box plot of sub-community transition frequency (i.e. for each node we measure the frequency at which the node changed sub-community membership over time), grouped by super-community;

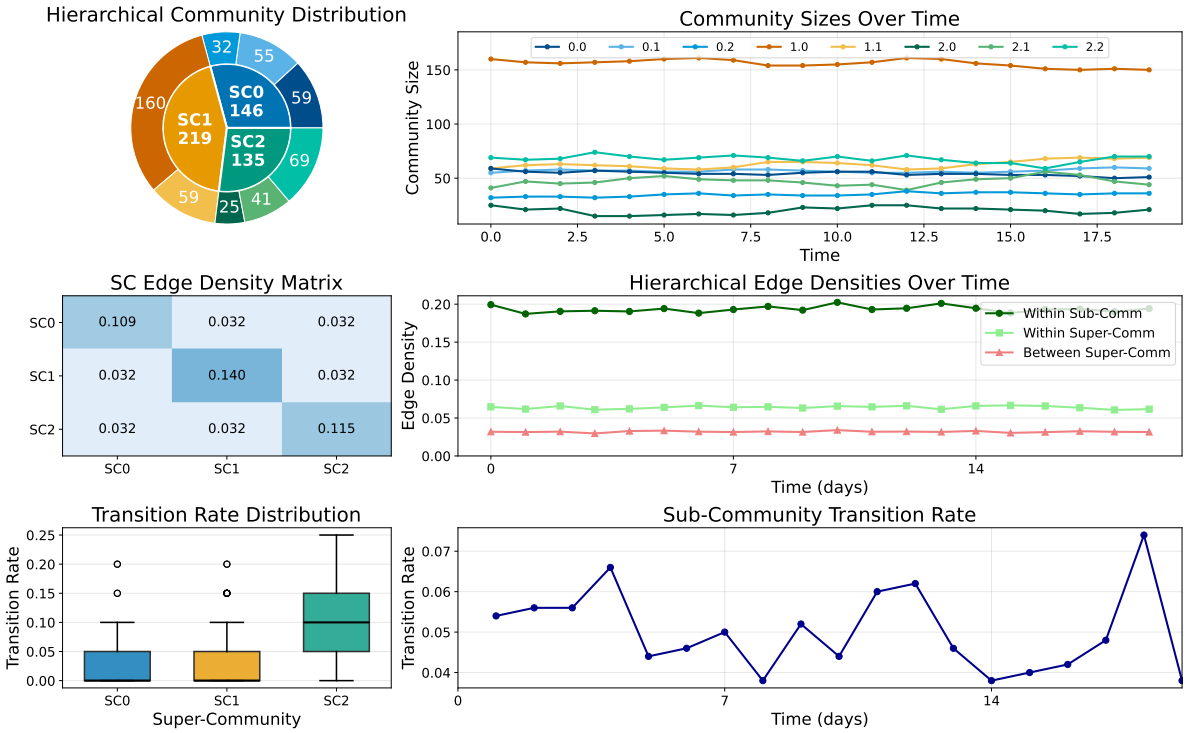

Figure 1: Overview of simulated dynamic hierarchical DCSBM. Refer to text for detailed explanation.

6. On bottom right, the overall transition rate, i.e. the ratio of nodes changing sub-community membership at each timestep $t > 0$.

This plot indicates for our simulated data, in order,

1. A split into three super communities, with 2,3 and 2 sub-communities, with split sizes in line with our probability vectors;

2. small movements in community size over time, consistent with our Markov model with high retention rate;

3. Consistent low edge frequency between super-communities (0.032), and slightly varying overall connectivity within super-communities of about 0.15, which aligns well with our choice of ρ_i[8]. The notably higher overall connectivity in SC1 results from it having only two sub-communities with a 70/30 size split.

[8]NB that each node has an individual beta distributed weight with mean approximately 0.8, hence all expected edge densities are 0.64 of the respective ρ_i values. The expected edge densities are hence 0.64 times $0.05, 0.1, 0.4$ which evaluates to 0.032, 0.064 and 0.256, and the within-super-community edge density is a weighted average of the latter two figures.

4. An almost constant time series of edge densities, consistent with the i.i.d. nature of individual weights and the non-directional nature of our Markov sub-community allocation, consistent in level with expected values,

5. A skewed distribution of community transition rate per super-community, variations in effective transition consistent with differences in sub-community split,

6. An overall transition rate that fluctuates between 0.095 and 0.125 over time, indicating a generally stable transition mechanism, in line with our Markov model.

We will use this visualization later on estimated community memberships on real world data.

3.3 Estimation algorithm

Our main goal is to establish an algorithm that assumes a DHDCSBM and effectively estimates the super-/sub-community structure and assignment per node from a given series of adjacency matrices.

In the case of $T = 1$ and SBMs, algorithms 1, 2 are consistent estimation methods, while for $R = 1$, we have algorithm 5 available. Our aim is to combine the ideas of these two algorithms to unlock analogue methods for community detection in this more general model. Loosely speaking, our idea is to first consider a UASE (or analogue using Laplacian) to generate estimated path embeddings $\mathbf{x}_i = (\mathbf{x}_i^t)_{t\in[T]}$, and cluster these into $\hat{K}$ estimated super-communities; then within each estimated super-community, we apply algorithm 5 to establish an estimator for sub-community mapping.

The main differences compared to the HSBM model is that our super-communities are differentiated from the sub-clusters not only by connection likelihood, but by virtue of temporal stability. A secondary difference is that we allow for degree correction, which requires some further considerations on normalization.

The main argument given in Lyzinski et al. (2016) for using a seeded algorithm over e.g. simple KMeans (compare Figure 1 and the discussion under Lemma 6) is that super-communities can include embeddings of varying magnitude but similar orientation, which can lead to a naive algorithm clustering sub-communities of smaller magnitude together,

even when those sub-communities belong to different super-communities. Since we will normalize the embeddings and consider whole path-wise clustering, these issues appear not to be too relevant for our case; indeed empirical testing showed no benefit for using a variant of algorithm 1 over standard clustering algorithms.

Before formulating our algorithms, we introduce some re-occurring notions. We aim to estimate τ from above in a two-step process, by estimating first a static super-community estimator $\hat{\tau}_{sup}$ with an estimated number $\hat{K}$ of clusters, and thereafter for each $k \in \hat{K}$ a dynamic sub-community estimator $\hat{\tau}^{sub}_{(k)}$. To make this more precise, if we are given a static super–community clustering $\hat{\tau}_{sup}$, we will then consider the split of our node index set $[n]$ into index sets

$$\hat{\mathcal{J}}_{(k)} := \hat{\tau}_{sup}^{-1}\{k\}$$

of size $\hat{n}_{(k)} := \left|\hat{\mathcal{J}}_{(k)}\right|$. We then designate $\hat{\gamma}_{(k)} : \hat{\mathcal{J}}_{(k)} \to [\hat{n}_{(k)}]$ as the enumeration mapping (i.e. the unique monotone bijection) with inverse $\hat{\gamma}^{-1}_{(k)}$, and for $\mathbf{A}^t$ as given adjacency matrices, we set

$$\mathbf{A}^t_{(k)} := \left(a_{\hat{\gamma}^{-1}_{(k)}(i),\hat{\gamma}^{-1}_{(k)}(j)}\right)_{i,j\in[\hat{n}_{(k)}]} , \tag{11}$$

the re-indexed reduction of $\mathbf{A}^t$ to the k-th super-community. (This is a formal description of the intuitive process of collecting the entries from $\mathbf{A}^t$ with $i, j \in \hat{\mathcal{J}}_{(k)}$, in order, into a new matrix $\mathbf{A}^t_{(k)}$.)

If we then construct time-dependent sub-community estimators $\hat{\tau}^{sub}_{(k)} : [\hat{n}_{(k)}] \times T \to [\hat{R}_{(k)}]$, we can combine this with the global estimator to

$$\hat{\tau}(i,t) := \sum_{k\in[\hat{K}]} \mathbf{1}[\hat{\tau}_{sup}(i) = k] \cdot \left(k,\ \hat{\tau}^{sub}_{(k)}\left(\hat{\gamma}_{(k)}(i), t\right) \right) . \tag{12}$$

(This is again a formal version of the intuitive process that, if i is predicted to be in in super community k, we set the first entry of our prediction to k for all t, and the second according to the time series of sub-indices predicted by $\hat{\tau}^{sub}_{(k)}$.)

We recall algorithm 5, which appears suitable for our 'inner loop' estimation of our sub-DCSBM. Based on experiments we found it advantageous to use a different dimensionality estimator, namely a regularized Akaike information criterion. We also found that using GMM can be very slow, and use KMeans to estimate the optimal cluster size using silhouette score. For sake of completeness we write out the modified algorithm 6. Here, the number of parameters correspond to dn parameters in $\mathbf{U}_d$, dnT parameters in

$\mathbf{V}_d$, and d^2 orthogonality constraints.

Algorithm 6: DCSBM community estimation

Data: Adjacency matrices $\mathbf{A}^1, \dots, \mathbf{A}^T$ of dimension $m \times m$, max dimension D, regularization strength λ

Result: Estimated embedding dimension $\hat{d}$, estimated number $\hat{K}$ of blocks, estimated block membership $\hat{\tau}(i,t) = r \in [\hat{K}]$ for node i and time t

Calculate Laplacians $\mathbf{L}^1, \dots, \mathbf{L}^T$ and unfold into $\mathbf{L} = \left[\mathbf{L}^1 | \cdots | \mathbf{L}^T\right]$;

for $d \in [D]$ **do**

Calculate $\mathbf{L}_d$ as rank d SVD approximation of $\mathbf{L}$;

Use $\mathbf{P}^t = \min\{1, \max\{0, \mathbf{L}_d^t\}\}$ element-wise;

Calculate log likelihoods

$$\ell(d) = \sum_{t \in [T]} \sum_{i<j} \left[l_{i,j}^t \log p_{i,j}^t + (1 - l_{i,j}^t)(1 - \log p_{i,j}^t) \right] \ ;$$

Calculate number of parameters

$$k_d := d \cdot n + d \cdot nT - d^2 \ ;$$

Assign $\mathrm{AIC}(d) = 2 \cdot (\lambda k_d - \ell(d))$;

Set $\hat{d} = \arg\max_d \mathrm{AIC}(d)$;

Apply algorithm 4 to obtain the rank $\hat{d}$ UASE $\hat{\mathbf{Y}} = \left[\hat{\mathbf{Y}}^1 | \cdots | \hat{\mathbf{Y}}^T\right]$ from $\mathbf{A}$;

Transform $\hat{\mathbf{Y}}$ into spherical coordinates $\hat{\mathbf{\Theta}} = \left[\hat{\mathbf{\Theta}}^1 | \cdots | \hat{\mathbf{\Theta}}^T\right]$ following (8);

Cluster the set $\left\{\hat{\theta}_i^t \ : \ i \in [m], t \in [T]\right\}$ in two steps: First use KMeans and silhouette score to estimate an optimal cluster size $\hat{R}$;

Then use GMM to estimate the cluster mapping $\hat{\tau} : [m] \times [T] \to \hat{R}$;

We are now ready to present a first (naive) algorithm for community estimation for DHDCSBM models:

This algorithm is simple and straightforward. However, it performs fairly poorly in practical tests against simulated data. One reason is that neither dimensioning nor clustering cares about the temporal stability of the clustering centres, and therefore dynamic assignments from the sub communities will easily contribute as noise.

Hence, we propose a modification to a (normalized) UASE, which we tentatively call a *temporally stabilized UASE*. To motivate this, let us say we have some D dimensional embedding $\hat{\mathbf{Y}} = \left[\hat{\mathbf{Y}}^1 | \cdots | \hat{\mathbf{Y}}^T\right]$, so that each $\hat{\mathbf{Y}}^t$ is a $n \times D$ matrix. If we fix $j \in [D]$, we can ask how much this dimension can contribute towards a time-invariant separation of

Algorithm 7: Simple community detection in dynamic hierarchical DCSBM

Data: Adjacency matrices $A^1, \dots, A^T$
Result: Estimated super-community/sub-community embeddings $\hat{\tau}(i,t) = (j,r)$
Compute normalized Laplacians $\mathbf{L}^t$ from $\mathbf{A}^t$, and unfold $\mathbf{L} = \left[\mathbf{L}^1 | \cdots | \mathbf{L}^T\right]$;
Calculate the SVD of $\mathbf{L} = \mathbf{U}\Sigma\mathbf{V}^\top$ and extract the singular values $s_1 \geq s_2 \geq \cdots$;
Estimate $\hat{D}$ as the elbow of the series s_i;
Construct the right hand dimension $\hat{D}$ spectral embedding $\hat{\mathbf{Y}} = \hat{\mathbf{Y}}_{\hat{D}}(\mathbf{L}) \in \mathbb{R}^{n \times T\hat{D}}$;
Use KMeans to estimate silhouette scores for clustering of the rows in $\hat{\mathbf{Y}}$, and choose $\hat{K}$ from the maximal silhouette score;
Use GMM to estimate *super-community clustering* $\hat{\tau}_{sup} : [n] \to [\hat{K}]$ for the rows of $\hat{\mathbf{Y}}$;
for $k \in [\hat{K}]$ **do**
  Define $\mathbf{A}^t_{(k)}$ according to (11), and unfold $\mathbf{A}_{(k)} = \left[\mathbf{A}^1_{(k)} | \cdots | \mathbf{A}^T_{(k)}\right]$;
  Use algorithm 6 to obtain a local estimator $\hat{\tau}^{sub}_{(k)}$ based on $\mathbf{A}_{(k)}$.
Combine $\hat{\tau}_{sub}$ and $\left(\hat{\tau}^{sub}_{(k)}\right)_{k \in [\hat{K}]}$ to a joint estimator $\hat{\tau}$ following (12).

node paths. To this end, we can define values $c_{t,i} := \hat{\mathbf{Y}}^t_{i,j}$, and interpret this as a matrix $\mathbf{C}(j)$ with indices time t and nodes i, and consider its *spatial variance*

$$\sigma^2_{\text{spatial},\delta} := \frac{1}{T} \sum_{t \in [T]} \text{Var}_i \left(c_{t,i}\right)$$

as a measure for signal strength (time-average of variation across relevant dimension $i \in [n]$), and its *temporal variance*

$$\sigma^2_{\text{temporal},j} := \frac{1}{n} \sum_{i \in [n]} \text{Var}_t \left(c_{t,i}\right)$$

as a measure for noise (node-average of variation across irrelevant dimension $t \in [T]$). This motivates to view the ratio

$$S_j := \frac{\sigma^2_{\text{spatial},j}}{\sigma^2_{\text{temporal},j}}$$

as a signal/noise ratio, measuring the capability of the embedding dimension j to establish temporally stable discrimination between nodes. We will hence score the embedding dimensions based on this measure, and apply an elbow cutoff to their sorted scores.

With this algorithm replacing our initial attempt, we are now ready to formulate our

Algorithm 8: Temporally stable projection

Data: Adjacency matrices $\mathbf{A}^1, \dots, \mathbf{A}^T \in \mathbb{R}^{n\times n}$, initial embedding dimension D
Result: Embedding matrix $\hat{\mathbf{X}} \in \mathbb{R}^{n\times T\hat{D}}$ of reduced dimension $\hat{D}$
Calculate normalized Laplacians $\mathbf{L}^t$ and unfold $\mathbf{L} = \left[\mathbf{L}^1|\cdots|\mathbf{L}^T\right]$;
Calculate $\hat{\mathbf{Y}}_D(\mathbf{L}) = \left[\hat{\mathbf{Y}}^1|\cdots|\hat{\mathbf{Y}}^T\right]$ from UASE applied to $\mathbf{L}$ with dimension D;
for $j \in [D]$ **do**
  Set $c_{t,i} = \hat{\mathbf{Y}}^t_{i,j}$ and compute

$$S_j := \frac{\frac{1}{T}\sum_{t\in[T]} \mathrm{Var}_i(c_{t,i})}{\frac{1}{n}\sum_{i\in[n]} \mathrm{Var}_t(c_{t,i})} ;$$

Sort dimensions as $j(1), \dots j(n)$ such that $S_{j(\cdot)}$ is decreasing;
Calculate cutoff value $\hat{D}$ using the elbow method on $S_{j(\cdot)}$;
for $t \in [T]$ **do**
  Set $\hat{\mathbf{X}}^t = \left(\hat{\mathbf{Y}}^t_{i,j(\nu)}\right)_{i\in[n],\nu\in[\hat{D}]} \in \mathbb{R}^{n\times\hat{D}}$;
Return $\hat{\mathbf{X}} = \left[\hat{\mathbf{X}}^1|\cdots|\hat{\mathbf{X}}^T\right]$;

temporally stable community estimation algorithm 9. I differs from algorithm 7 in the initial embedding estimation.

Algorithm 9: Community detection in hierarchical dynamic DCSBM

Data: Adjacency matrices $\mathbf{A}^1, \dots, \mathbf{A}^T$, initial dimension D
Result: Estimated super-community/sub-community embeddings $\hat{\tau}(i,t) = (j,\nu)$
Use Algorithm 8 with $\mathbf{A}$ and initial dimension D to calculate temporally stable embeddings $\hat{\mathbf{X}} \in \mathbb{R}^{n\times\hat{D}T}$;
Use KMeans to estimate a *global clustering* $\hat{\tau}_{sup} : [n] \to [\hat{K}]$ for the rows of $\hat{\mathbf{X}}$, with optimal cluster size $\hat{K}$ estimated from silhouette scores;
for $k \in [\hat{K}]$ **do**
  Define $\mathbf{A}^t_{(k)}$ according to (11), and unfold $\mathbf{A}_{(k)} = \left[\mathbf{A}^1_{(k)}|\cdots|\mathbf{A}^T_{(k)}\right]$;
  Use algorithm 6 to obtain a local estimator $\hat{\tau}^{sub}_{(k)}$ based on $\mathbf{A}_{(k)}$.
Combine $\hat{\tau}_{sub}$ and $\hat{\tau}^{sub}_{(k)}$ to a joint estimator following (12).

4 Results

4.1 Estimation results on simulated data

Our main application on simulation results consists of testing the performance of our communities estimation results with different sets of parameters. To this end, we use the adjusted rand index as basis (see (Murphy, 2012, Section 25.1.2.2)). However, we wish to differentiate our classification performance on both super- and sub-communities, one of which being static while the other dynamic. To this end, we use two ARI scores.

The first score, called "super ARI", measures the ARI of the static assignment to super-communities. The second score, "mean ARI", measures the ARI of dynamic assignments to sub-communities and merits some explanation. In order to get a simple global label of all sub-communities, we apply a tiered label to them. For instance, in the case of three super communities with $2, 3$ and 2 sub-communities each, we can use the strings $1.1, 1.2, 2.1, 2.2, 2.3, 3.1, 3.2$ to identify all seven sub-communities. For both ground truth and estimated super- and sub-community assignments, one can thus derive categorical labels and assignments of pairs (i, t) to either ground truth or estimated label. This approach can be applied also when the estimated number of super-communities differs from the ground truth, in contrast to more structured solutions.

We will calculate the ARI of these "de-hierarchized" sub-community assignments separately for each t (reflecting the quality of clustering in a snapshot), and report the time average of these ARI as *mean ARI*, a measure of the quality of sub-community assignment.

For simulation, we chose a default set of parameters and varied consecutively a single parameter (or set of tightly connected parameters) through a given range. Table 1 summarizes the default and range for our parameters. We repeated each simulation for $N = 10$ simulations and recorded average and standard deviation for super and mean ARI, as well as run time in seconds. The code for this is found in the notebook `ari_test_notebook.ipynb`.

We will first compare performance figures for selected parameter sets for the two algorithms, as shown in Table 2. We show the parameter deviating from the default value from Table 1, and then show mean and standard deviation for super and mean ARI, for simple and temporally stable estimator (algorithms 7 and 9) respectively.

Table 1: Parameter Default Values and Ranges

Experiment	**Parameter**	**Default Value**	**Value Range**
Varying n	n	1000	200 500 1000 1500 2000
Varying T	T	20	5 10 20 30 50
Varying ρ difference	(ρ_1, ρ_2, ρ_2)	(0.05, 0.1, 0.4)	$(0.05, 0.1, 0.15)$ $(0.05, 0.1, 0.25)$ $(0.05, 0.1, 0.4)$ $(0.05, 0.1, 0.6)$ $(0.05, 0.1, 0.9)$ $(0.05, 0.1, 0.95)$
Varying ρ magnitude	(ρ_1, ρ_2, ρ_2)	(0.05, 0.1, 0.4)	$(0.005, 0.01, 0.04)$ $(0.010, 0.02, 0.08)$ $(0.025, 0.05, 0.2)$ $(0.05, 0.1, 0.4)$ $(0.1, 0.2, 0.8)$
Varying stay probabilities	p_{stay}	(0.2, 0.2, 0.2)	$(0.2, 0.2, 0.2)$ $(0.4, 0.4, 0.4)$ $(0.6, 0.6, 0.6)$ $(0.8, 0.8, 0.8)$ $(0.9, 0.9, 0.9)$ $(0.95, 0.95, 0.95)$
Number of super communities	K	3	-
Probability vector	π	$[0.3, 0.45, 0.25]$	-
Number of sub communities	$R_{(k)}$	$[3, 2, 3]$	-
Sub community probabilities	$\Pi = (\pi_{(k)})_{k\in[K]}$	$[0.3, 0.45, 0.25]$ $[0.7, 0.3]$ $[0.2, 0.3, 0.5]$	-
Stay probabilities	$p_{(k),stay}$	$[0.95, 0.9, 0.85]$	-
Weight Beta parameters	$((\alpha_{(k)}, \beta_{(k)}))_{k\in[K]}$	$(8, 2)$ $(8, 2)$ $(8, 2)$	-

Table 2: Performance of simple and stable algorithm on different parameters

Algorithm	Parameters	super ARI		mean ARI	
		Mean	SD	Mean	SD
Simple	$n = 500$	0.57	0.18	0.53	0.14
	$n = 2000$	0.68	0.17	0.70	0.08
	$T = 10$	0.50	0.20	0.57	0.12
	$T = 30$	0.67	0.17	0.62	0.11
Stable	$n = 500$	0.97	0.06	0.75	0.05
	$n = 2000$	1.00	0.00	0.87	0.04
	$T = 10$	1.00	0.00	0.91	0.01
	$T = 30$	1.00	0.00	0.80	0.02

While the simple algorithm performs clearly better than random (ARI=0), temporal stabilization improves both super and mean ARI by a surprisingly large margin; super ARI achieves almost perfect scores already on relatively small datasets, and mean ARI increases to high levels with still manageable network sizes. Moreover, the ARIs for the simple algorithm are not only low but show no definitive sign of improvement upon increasing the number of nodes or time horizon.

This confirms our first major empirical result, namely that our temporal stability scoring is an effective method for identifying temporally stable dimensions which leads to efficient static clustering of our dynamic networks.

We now study chart 2 which shows a summary of super/mean ARI when varying different parameters, using algorithm 9. This also depicts a plot of the average run times of two sets (varying n and varying T), with the run time in seconds plotted against the number of data points $Tn(n-1)/2$, as a log/log chart. Interestingly, this plot shows that increasing T leads to a larger increase in run time, relative to the number of data points, than increasing n.

We consider now the performance on varying n and T. We see that super ARI increases rapidly, and mean ARI reasonably, with increasing n. However, we notice that an increase of T beyond a 10 seems to slightly *decrease* the mean ARI, despite an increase in available data. This is a somewhat surprising (if relatively mild) effect, which could indicate that our dynamic DCSBM algorithm might not be set up to extract information optimally.

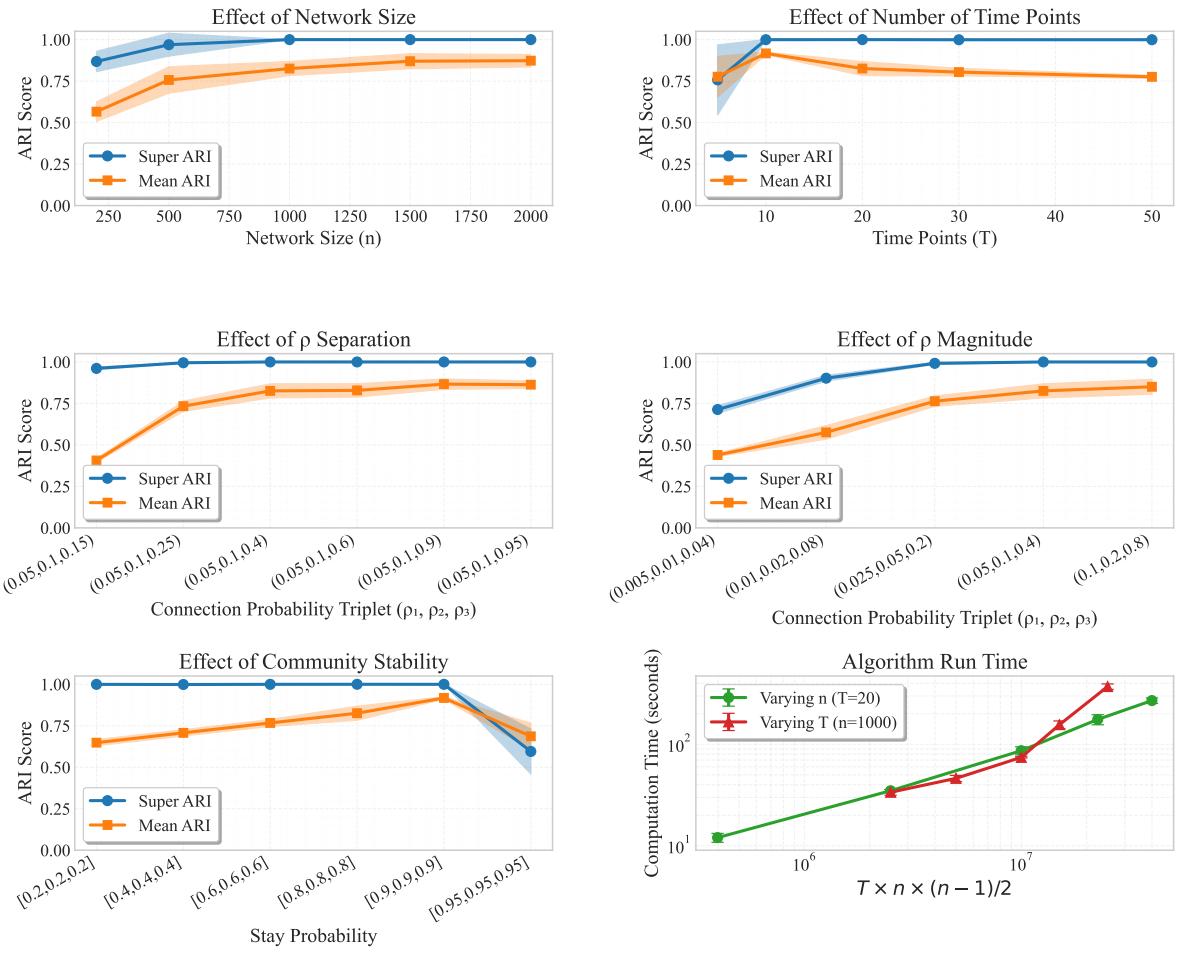

Figure 2: Super and mean ARI of algorithm 9 on varying sets of parameters. Empirical mean shown as markers, empirical standard deviation as shaded areas.

On ρ separation, we see a clear if slowing improvement on mean ARI over the parameter range. We note for completeness' sake that in a previous variant of the experiment run without proper care on reproducibility, we observed a clear reduction in performance for $\rho_3 = 0.9$, which the final version of experiment does not reproduce.

The effect of overall node density (ρ magnitude) is mostly as hoped, a steady increase in super and mean ARI with overall reduction in standard deviation.

Maybe most interesting is the behaviour of super/mean ARI upon changing the sub-community stability p_{stay} (the likelihood of staying in the same sub-community from t to $t+1$ instead of finding a random new one). Initially, super ARI stays unaffected at near perfect levels, while the mean ARI initially increases with increasing p_{stay}. This is intuitive if we recall that UAE yields asymptotically static estimators for static community assignments. However, for $p_{stay} = 0.95$, we suddenly see a strong reduction of super ARI and also some decrease in mean ARI. Inspection of the ten runs confirms this to be systematic as 9 out of 10 runs score a super ARI below 0.56. Here we see the downside of our focus on temporal stability alone as criterion for super-community cluster dimensions; temporally stable sub-communities can be mis-identified as super-community clusters.

In summary, we find that our temporally stabilized estimation method is capable to reproduce the structure and community membership of DHDCSBM models with a moderate to high degree of precision, depending on parameter choice. Most dependencies on parameters are natural, however the model notably fails to infer the hierarchical structure when both super- and sub-community membership are stable over time.

4.2 Estimation results on Cycling Infrastructure Data

As a practial application, we apply our temporally stabilized algorithm to some 9 real world data. We consider a data set from Transport for London's Cycling Infrastructure Data set (Transport for London, 2025). From `https://cycling.data.tfl.gov.uk/` under `usage-stats`, we used the latest monthly data for May 2025 (split into two csv files `419JourneyDataExtract01May2025-14May2025.csv` and `420JourneyDataExtract14May2025-31May2` which we joined). The raw data consists of about 830 000 travels of labeled bicycles

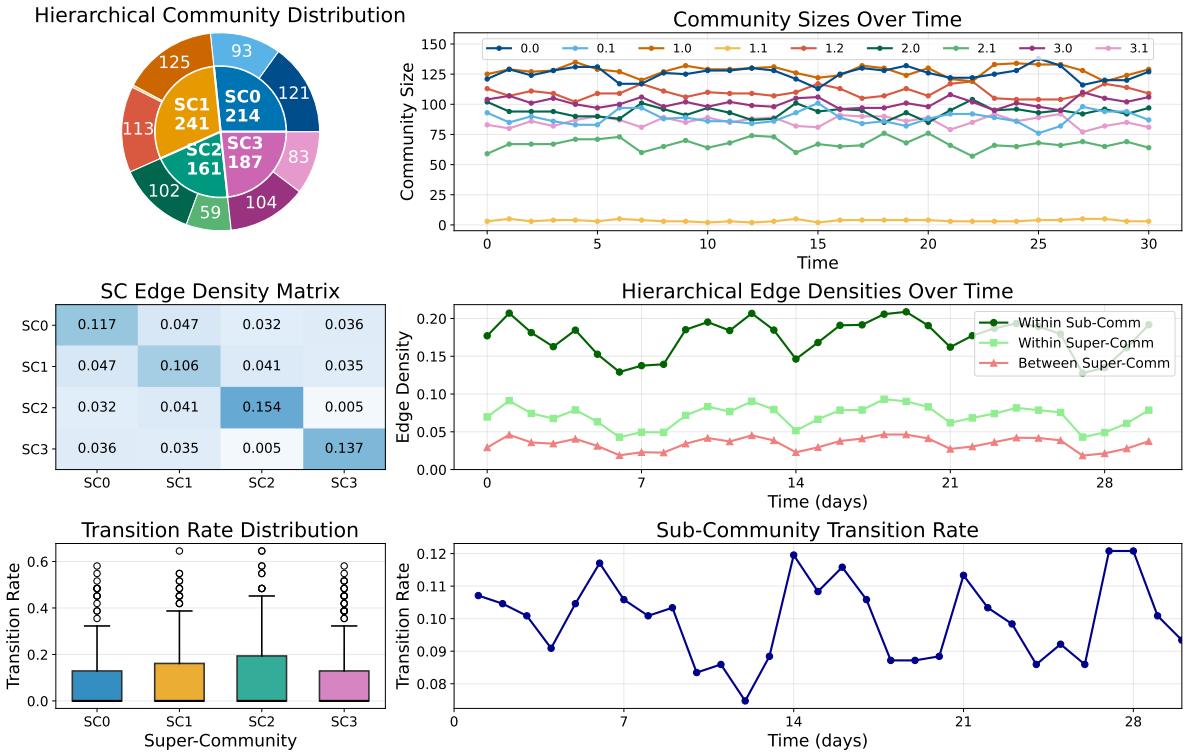

Figure 3: Dashboard of estimated super-/sub-communities in the bike sharing dataset. Setup as explained in subsection 3.2

between 803 individual stations in Greater London, with time stamps on start and end time for each journey.

We processed the data by first removing about 30 000 journeys where start and end station coincide, and then grouped the data into daily connections based on the day of journey start. For each day t we then connected stations i and j in the (symmetric) adjacency matrix $\mathbf{A}^t$ if at least one journey between the stations was registered on this day. This eliminates multiple journeys per day, and leaves us with 570 000 edges between 803 nodes spread over 31 days. The average (node and time) degree is 45.7, and the average edge density is 0.057.

The resulting list of adjacency matrices was then run through our algorithm, which selected three dimensions (dimensions 2-4) from UASE based on the elbow criterion for the temporal stability score, and estimated four super-communities. Each super-community was then split further into two sub-communities, except the second which was split in three (one of which only consisting of 3 nodes).

We summarize the results in figure 3, the setup of which was discussed in detail in

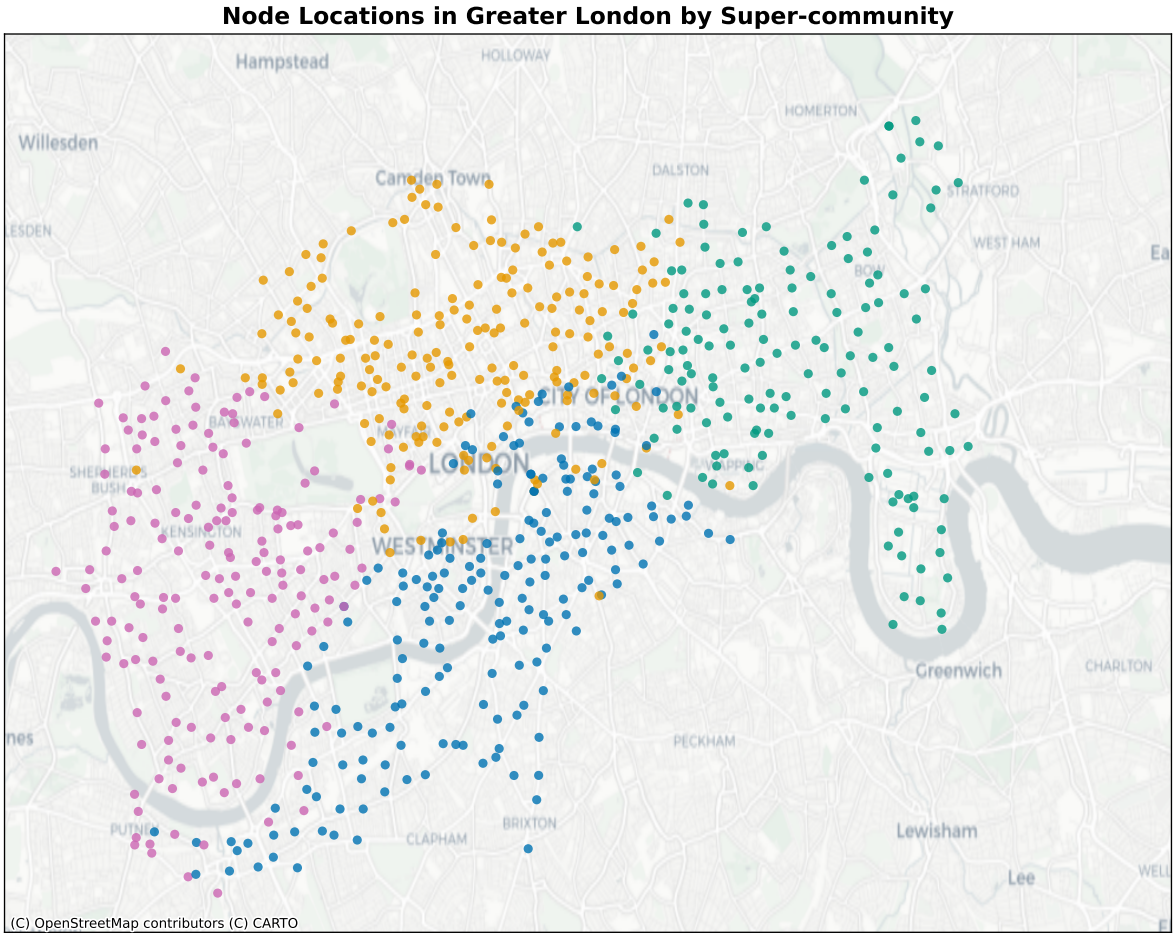

Figure 4: Super community mapping in geospatial coordinates.
Map tiles from Carto, under CC BY 3.0. Data by OpenStreeMap, under ODbL. Latitude/longitude data from Transport for London

subsection 3.2. The sizes of super-communities range from 161 to 241, and we see inter-super community edge densities ranging from 0.005 to 0.05 while intra-super-community edge densities are clearly higher (between 0.11 and 0.15). We can also note a weekly cyclical pattern in hierarchical edge densities over time, expressed uniformly over all three levels of edge counts.

The sub-communities evolve over time in a quite stable fashion. Sub-community sizes vary little over time, and the transition rate fluctuates between 0.08 and 0.12, indicating an overall high and persistent temporal stability. A look at the sub-community transition rate distribution shows a highly skewed node-wise distribution of transition rates. Some nodes are highly stable, while others seem to change often between sub-communities.

In figure 4, 793 of the 803 nodes are mapped with geospatial locations of stations, col-

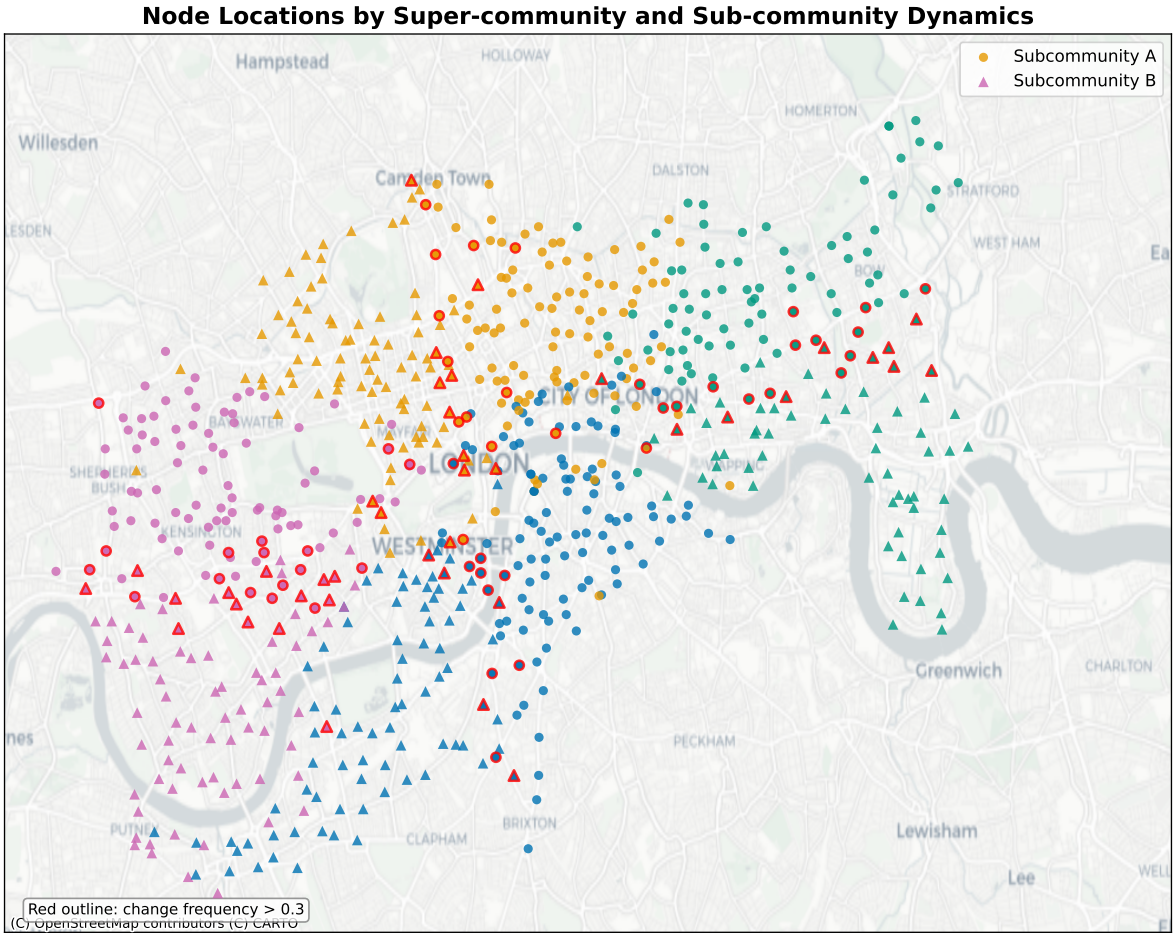

Figure 5: Super-/sub community mapping in geospatial coordinates. Shape indicates majority (mode) of sub community assignment, per super community. Red circumference indicates a transition frequency above 0.3.
Map tiles from Carto, under CC BY 3.0. Data by OpenStreeMap, under ODbL. Latitude/longitude data from Transport for London

ored by super-community, using latitude and longitude data obtained from Transport for London (2025). The super-communities delineate mostly clear geographic boundaries (roughly East, West, North and South), a notable exception being the City of London where East, North and South share membership. The Thames is a partial separator, with the notable exception of Westminster which is assigned to the South super-community. This dissection is quite similar to information-theoretic clustering results (compare e.g. with the Louvian clustering in Figure 2 in Munoz-Mendez et al. (2018)).

Regarding sub communities, we refer to figure 5[9]. We use the same colors as before for super-communities, but assign a shape based on the predominant sub-community of each node; furthermore nodes with an average transition rate above 0.3 are shown with a red circumference. We see a clear spatial separation of all majority sub-communities, and notice that nodes with high transition frequency populate the borderlines between sub-communities.

In summary, our model estimation leads to a spatial clustering picture consistent with geospatial location and similar to information-theoretic clusters. Meanwhile, temporal variations in edge density permeate through our community hierarchy (3, mid right). This indicates that our model at least in large parts separates this variation from the community structure. While these results are not particularly surprising, our model was able to extract meaningful latent model structure from the adjacency matrices.

5 Discussion

We introduced a new model of dynamic hierarchical networks which extends previous models for random graphs and adds a new aspect by imposing a static assignment to super-communities while allowing for a dynamic assignment to sub-communities. Further, we established an estimation algorithm for super- and sub-community assignment inspired by both the established hierarchical random graph estimation and the recent introduction of the unfolded adjacency embedding. By adding a novel component to control for temporal stability of embeddings, we ended up with an algorithm that achieved good to excellent performance on simulated data. An important shortcoming appeared when sub-community assignment was set as mostly stationary, in which case our algorithm fails to differentiate between super- and sub-communities.

This estimator was also applied to real world data from the TfL CID dataset, and produced a reasonable split of the 803 stations with a clear distinction in connectivity along the hierarchy. Visible seasonable patterns in overall connectivity were reflected evenly in the hierarchically classified edge types, and did not impact the (rather stable) sub-community sizes. The super- and sub-community clusters are strongly aligned with

[9]For simplicity, we assigned the 3 node sub-community in the East super-community to the majority sub-community, which leaves us with two sub-communities per super-community.

obvious geo-spatial patterns, indicating that our algorithm is capable of extracting latent features from connectivity data.

However, we have also seen some clear limitations of our approach. One limitation is that the algorithm clusters super-communities in isolation, without regard for the total fit of the complete method. While the adjustment for temporal stability has proven empirically to be an effective detection method for stable community assignment, its general lack of 'hierarchical awareness' is visible in the performance deterioration when pushing the sub-communities towards being almost stable.

A second and less expected shortcoming is the visible performance deterioration when applying the algorithm to longer time intervals. Since in our example the super-ARI is close to perfect, this issue appears to be connected to a weakness in our dynamic DCSBM estimator. The main suspect here would be the UASE itself, as the remainder essentially clusters larger and larger datasets in a fixed dimension. We were not able to investigate this phenomenon more thoroughly in the given time period.

In practical terms, our model's reliance on UASE is very convenient in that it avoids complex procedures like Procrastes algorithms to align estimates over time. This does however also bind it to singular value decomposition in high and ever increasing dimensions, which has a clear impact on run time. The run time performance of our estimators as they are do not scale well to long time periods. In general, the algorithms as presented would require additional work to prepare them for very large networks like social or bitcoin transaction networks.

One strength of the UAE driven estimation of dynamic membership is its relative indifference to the time structure of the generating model. However, this also implies that we do not directly derive information about it in our estimation. A rough first impression from our bike sharing data implies a clear heterogeneity in the dynamic of sub-communities where some nodes change membership often and others never. Establishing temporal dynamics for membership evolution appears to be an important venue which is not covered by our results.

Our model is well suited to describe a specific kind of dynamic network behaviour, where larger static clusters split into smaller clusters with dynamic membership, and is in particular set up for *hierarchically homophilic* edge connectivity. These are however fairly specific assumptions which will likely limit its practical applicability. However, the new method of detecting temporally stable clusters might prove to have broader potential application.

Finally, we did not manage to establish any theoretical results regarding asymptotic consistency. Further research into this topic could in turn yield insight into improvements to our algorithms.

6 Endmatter

All data used is publicly available, under both the web resource https://cycling.data.tfl.gov.uk/ and in the `data` folder in the github repository Creutzig (2025).

The code used to produce all simulations and estimations can be found in the github repository as well. The simulations and their estimators were run using the jupyter notebook `ari_test_notebook.ipynb`. The TfL CID data was first manually downloaded and placed in the `data` folder, then processed using the `process_TfL_CID.ipynb` notebook, and estimated using the `bike_data_estimation.ipynb` notebook.

References

Abbe, E. (2018). Community detection and stochastic block models: recent developments. *Journal of Machine Learning Research*, 18(177):1–86.

Creutzig, J. (2025). Thesis repository. https://github.com/jcreutzig/thesis.

Gallagher, I., Jones, A., and Rubin-Delanchy, P. (2021). Spectral embedding for dynamic networks with stability guarantees. *Advances in Neural Information Processing Systems*, 34:10158–10170.

Hoff, P. D., Raftery, A. E., and Handcock, M. S. (2002). Latent space approaches to social network analysis. *Journal of the american Statistical association*, 97(460):1090–1098.

Humphries, M. D. (2017). Dynamical networks: Finding, measuring, and tracking neural population activity using network science. *Network Neuroscience*, 1(4):324–338.

Jia, X., Siegle, J. H., et al. (2022). Multi-regional module-based signal transmission in mouse visual cortex. *Neuron*, 110(9):1585–1598.

Jones, A. and Rubin-Delanchy, P. (2020). The multilayer random dot product graph. *arXiv preprint arXiv:2007.10455*.

Karrer, B. and Newman, M. E. (2011). Stochastic blockmodels and community structure in networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics*, 83(1):016107.

Kim, B., Lee, K. H., Xue, L., and Niu, X. (2018). A review of dynamic network models with latent variables. *Statistics surveys*, 12:105.

Lee, C. and Wilkinson, D. J. (2019). A review of stochastic block models and extensions for graph clustering. *Applied Network Science*, 4(1):1–50.

Lyzinski, V., Tang, M., Athreya, A., Park, Y., and Priebe, C. E. (2016). Community detection and classification in hierarchical stochastic blockmodels. *IEEE Transactions on Network Science and Engineering*, 4(1):13–26.

Munoz-Mendez, F., Han, K., Klemmer, K., and Jarvis, S. (2018). Community structures, interactions and dynamics in london's bicycle sharing network. In *Proceedings of the 2018 ACM international joint conference and 2018 international symposium on pervasive and ubiquitous computing and wearable computers*, pages 1015–1023.

Murphy, K. P. (2012). *Machine learning: a probabilistic perspective*. MIT press.

Nickel, C. L. M. (2008). *Random dot product graphs a model for social networks*. PhD thesis, Johns Hopkins University.

Rubin-Delanchy, P., Cape, J., Tang, M., and Priebe, C. E. (2022). A statistical interpretation of spectral embedding: the generalised random dot product graph. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 84(4):1446–1473.

Sanna Passino, F., Bertiger, A. S., Neil, J. C., and Heard, N. A. (2021). Link prediction in dynamic networks using random dot product graphs. *Data Mining and Knowledge Discovery*, 35(5):2168–2199.

Sanna Passino, F., Heard, N. A., and Rubin-Delanchy, P. (2022). Spectral clustering on spherical coordinates under the degree-corrected stochastic blockmodel. *Technometrics*, 64(3):346–357.

Sarkar, P. and Moore, A. W. (2005). Dynamic social network analysis using latent space models. *Acm sigkdd explorations newsletter*, 7(2):31–40.

Siegle, J. H., Jia, X., et al. (2021). Survey of spiking in the mouse visual system reveals functional hierarchy. *Nature*, 592(7852):86–92.

Transport for London (2025). Cycling Infrastructure Database. https://cycling.data.tfl.gov.uk/.

Young, S. J. and Scheinerman, E. R. (2007). Random dot product graph models for social networks. In *International Workshop on Algorithms and Models for the Web-Graph*, pages 138–149. Springer.